

第6章 C++工具

前面重点介绍了 C++语言的封装性、继承性、重载及多态性。本章主要介绍 C++的一些编程工具。这些工具可以帮助程序设计人员更方便地进行程序的设计和调试工作，提高程序开发效率。在 C++发展的后期，有些 C++编译系统根据实际工作的需要，采用工具形式增加了一些功能。主要包括函数模板和类模板、异常处理、命名空间和运行时类型识别。后来 ANSI C++委员会将它们纳入到 ANSI C++标准中，并建议所有的 C++编译系统都能实现这些功能。

6.1 函数模板和类模板

C++提供的工具之一就是函数模板和类模板，而且两者的定义方法也很相似。模板是 C++支持多态性的一种工具，体现 C++泛化编程思想；所谓模板，其实就是一种使用“数据类型”作为参数来产生一系列函数或类的机制。因此，模板是通过使用“类型参数”来完成不同的功能。使用模板可以让用户得到类或函数声明的一种通用模式。模板方便了更大规模的软件开发。减少了程序员编写代码的工作量。下面分别介绍函数模板和类模板。

6.1.1 函数模板

C++提供的函数模板(function template)工具，实际上是建立一个通用函数，其函数类型和形参类型不具体指定，用一个虚拟的类型来代表。这个通用函数就称为函数模板。凡是函数体相同的函数都可以用这个模板来代替，不必定义多个函数，只需在模板中定义一次即可。在调用函数时系统会根据实参的类型来取代模板中的虚拟类型，从而实现了不同函数的功能。

实例 6-1 通过函数模板来实现不同类型数据的比较函数。

```
#include <iostream>
using namespace std;
template<typename T> //模板声明，其中T为类型参数
T max(T a, T b, T c) //定义一个通用函数，用T作虚拟的类型名
{
    if(b>a) a=b;
    if(c>a) a=c;
    return a;
}
```

```

int main( )
{
    int i1=185, i2=-76, i3=567, i;
    double d1=56.87, d2=90.23, d3=-3214.78, d;
    long g1=67854, g2=-912456, g3=673456, g;
    i=max(i1, i2, i3); //调用模板函数, 此时T被int取代
    d=max(d1, d2, d3); //调用模板函数, 此时T被double取代
    g=max(g1, g2, g3); //调用模板函数, 此时T被long取代
    cout<<"i_max="<<i<<endl;
    cout<<"d_max="<<d<<endl;
    cout<<"g_max="<<g<<endl;
    return 0;
}

```

在程序实例 6-1 中, 语句 4-9 定义了一个 2 个数据比较的函数模板 `max()`。在模板中, 函数参数的数据类型, 以及函数返回值的数据类型都是用虚拟类型符号 `T` 来表示。在主函数 `main()` 中, 语句 16-18 分别 3 次调用函数模板 `max()`, 并分别用实际数据类型 (`int`, `double`, `long`) 来替换模板的虚拟类型 `T`。例如, 在对程序进行编译时, 遇到第 16 行调用函数 `max(i1, i2, i3)`, 编译系统会将函数名 `max()` 与模板 `max()` 相匹配, 将实参的类型取代了函数模板中的虚拟类型 `T`。因此, 语句 4-9 称为函数模板的定义语句, 而语句 16-18 称为函数模板的调用语句。因此, 使用函数模板的方法可以归纳为如下 2 个过程。

- (1) 说明和定义函数模板;
- (2) 对函数模版实例化, 即通过实际类型参数来替换虚拟类型参数, 将函数模版实例化为相应的模板函数, 并调用和执行模板函数。

通过上述程序分析, 可以归纳出函数模板的定义一般形式:

```

template<class T1, class T2...>
{
    通用函数体定义
}

```

其中, `T1`, `T2`……是函数模板中的虚拟类型名; `class` 是关键词。

函数模板的实例化过程就是采用具体实际类型参数来调用函数模板。当编译系统发现有一个对应的函数调用时, 将根据实参中的数据类型来确认是否匹配函数模板中对应的形参, 然后生成一个模版函数。

函数模版定义和实例化的过程中, 需要注意如下几个问题。

- (1) 函数模板的申明和定义必须在全局作用域。因此, 函数模板不能说明为类的成员函数。
- (2) 模板类型参数不具有隐式 (自动) 类型转换的作用; 类型必须完全匹配。
- (3) 模板函数也可以重载。匹配过程有以下规定: 首先匹配类型完全相同的重载函数。其次, 才寻求函数模板来匹配。

下面通过程序实例来说明上述过程及相关注意事项。

实例6-2

```
#include <iostream>
using namespace std;

int GetMax( int a, int b ) //求两个整型数的最大值
{   cout << "调用int, maxValue = ";
    return ( a > b ) ? a : b;}

long GetMax( long a, long b ) //求两个长整型数的最大值
{   cout << "调用long, maxValue = ";
    return ( a > b ) ? a : b;}

double GetMax( double a, double b ) //求两个双精度型数的最大值
{   cout << "调用double, maxValue = ";
    return ( a > b ) ? a : b;}

//char GetMax( char a, char b ) //求两个字符型数的最大值
//{   cout << "调用char, maxValue = ";
//    return ( a > b ) ? a : b;}

//定义函数模板,在函数模板中添加第三参数char *,是为了与函数模板区分开
template <class Type>
Type GetMax( Type a[ ], int iCnt, char *lpszArrayName )
{   int i;
    Type tMaxValue = a[0]; //定义Type类型的变量
    for ( i=1; i<iCnt; i++ ) //在循环中寻找数组中最大的值
    {   if ( tMaxValue < a[i] )
        {tMaxValue = a[i];    }
    }
    cout << "使用函数模板, " << lpszArrayName << "的最大值, maxValue = ";
    return tMaxValue;}

//定义函数模板
template <class TypeX, class TypeY>
TypeX GetMax( TypeX tX, TypeY tY )
{   TypeX tMaxValue = 0; //定义一个TypeX类型变量
    if ( tX > ( TypeX )tY ) //比较前, 首先将TypeY类型变量转化为TypeX类型变量
    {tMaxValue = tX;}
    else
    {tMaxValue = ( TypeX )tY;}
    cout << "调用函数模板, maxValue = ";
    return tMaxValue;}

void main( )
{   int a[ ] = {1, 3, 5, 7, 9, 6, 4, 8, 2, 10};
    double b[ ] = {3.2, -6.4, 6.0, 9.9, 8.6, 2.1};
```

```

char c[ ] = {'A', 'C', '1', 'a', 'c'};

cout << "      " << GetMax( a, 10, "数组a" ) << endl; //使用函数模板
cout << "      " << GetMax( b, 5, "数组b" ) << endl; //使用函数模板
cout << "      " << GetMax( c, 5, "数组c" ) << endl; //使用函数模板

cout << "      " << GetMax( 10, 20 ) << endl; //调用重载函数
cout << "      " << GetMax( 101L, 201L ) << endl; //调用重载函数
cout << "      " << GetMax( 1.0, 2.0 ) << endl; //调用重载函数

cout << "char=" << GetMax( 'A', '2' ) << endl; //使用函数模板

cout << "      " << GetMax( 10, 5.0 ) << endl; //使用函数模板
cout << "      " << GetMax( 11.1, 5 ) << endl; //使用函数模板
cout << "      " << GetMax( 22, 10L ) << endl; //使用函数模板
cout << "      " << GetMax( 'A', 2L ) << endl; //使用函数模板
cout << "      " << GetMax( 1.0, 200L ) << endl; //使用函数模板
cout << "      " << GetMax( 100.0, 'A' ) << endl; //使用函数模板

system("pause");
}

```

该程序运行结果为：

```

使用函数模板，数组 a 的最大值，maxValue =      10
使用函数模板，数组 b 的最大值，maxValue =      9.9
使用函数模板，数组 c 的最大值，maxValue =      c
调用 int，maxValue =      20
调用 long，maxValue =      201
调用 double，maxValue =      2
调用函数模板，maxValue = char=A
调用函数模板，maxValue =      10
调用函数模板，maxValue =      11.1
调用函数模板，maxValue =      22
调用函数模板，maxValue =      A
调用函数模板，maxValue =      200
调用函数模板，maxValue =      100

```

通过分析上述程序运行结果可知：GetMax(10, 20)直接调用 int GetMax(int a, int b)，而没有匹配模板，因为遵循重载函数优先的原则；同理，如果没有注释掉 char GetMax(char a, char b)以前，GetMax('A' , '2')调用该重载函数；但注释掉 char GetMax(char a, char b)以后， GetMax('A' , '2')调用了模板函数 2。这就说明在匹配模板函数时，系统不会

进行隐式类型转换以匹配重载函数，否则它就应该调用 `int GetMax(int a, int b)`。另外，`GetMax(10, 5.0)`调用函数模板 2 时，在重载函数中没有匹配版本，而是在函数模板中匹配，调用了函数模板 2。

6.1.2 类模板

前面介绍了 C++语言的函数模板的概念。同理，C++还提供了类模板机制。如果有若干个类的功能相同，仅仅是类中的数据成员的类型不同，则可以声明一个通用的类模板。或者说，类模板可使类中的某些数据成员、成员函数的参数，或返回值能取任意类型。因此，如果说类是对象的抽象，对象是类的实例，则类模板是类的抽象，类是类模板的实例。类模板也称为“参数化类”。类模板也包括类模板申明、以及类模板的实例化 2 个阶段。

1. 类模板的申明

类模板申明的一般规则为：在类申明前面加入一行语句，格式如下。

`template<class 虚拟类型参数>`，如

`class 模板名(如：Compare)`

`{类体...};`

例如，下面就是类模板例子。

```
template<class T>
class Compare
{public :
Compare(T a,T b)
{x=a;y=b;}
T max( )
{return (x>y)?x:y;}
T min( )
{return (x<y)?x:y;}
private :
T x,y;
};
```

2. 类模板的实例化

在申明了通用类模板之后，可以利用类模板来建立含各种数据类型的实际的类，这个实际的类也称为模板类。然后通过这些实际模板类来构建具体对象。这个过程称为类模板的实例化。下面结合上述 Compare 类模板来介绍类模板实例化的过程。

Compare 是一个类模板的名称，而不是一个具体的类，类模板体中的类型 T 并不是一个实际的类型，只是一个虚拟的类型。因此，在类实例化时，必须用实际类型名去取代虚拟的类型，具体的做法如下。

```
Compare <int> cmp(4,7);
```

即在类模板名之后在尖括号内指定实际的类型名，在进行编译时，编译系统就用 `int` 取代类模板中的类型参数 `T`，这样就把类模板具体化了，或者说实例化了。因此，可以将类模板的实例化方法总结如下一般格式。

类模板名<实际类型名> 对象名；

或：类模板名<实际类型名> 对象名(实参表列)；

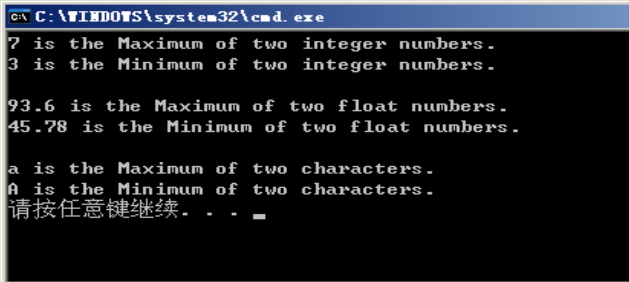
例如：`Compare<int> cmp;`

或 `Compare<float> cmp(3.1, 7.2);`

上述分别采用 `int` 和 `float` 实际数据类型对类模板 `Compare` 进行实例化，同时构建了 2 个对象 `cmp` 和 `cmp(3.1, 7.5)`。

综合上述类模板的 2 个过程，下面列举了一个完整的类模板程序。程序实例 6-3，类模板的申明与实例化。

```
1  #include <iostream>
2  using namespace std;
3  template<class numtype>           //定义类模板
4  class Compare
5  {public:
6  Compare(numtype a,numtype b)
7  {x=a;y=b;}
8  numtype max( )
9  {return (x>y)?x:y;}
10 numtype min( )
11 {return (x<y)?x:y;}
12 private:
13 numtype x,y;};
14
15 int main( )
16 {Compare<int> cmp1(3,7);          //定义对象cmp1，用于两个整数的比较
17 cout<<cmp1.max( )<<" is the Maximum of two integer numbers."<<endl;
18 cout<<cmp1.min( )<<" is the Minimum of two integer numbers."<<endl<<endl;
19 Compare<float> cmp2(45.78,93.6); //定义对象cmp2，用于两个浮点数的比较
20 cout<<cmp2.max( )<<" is the Maximum of two float numbers."<<endl;
21 cout<<cmp2.min( )<<" is the Minimum of two float numbers."<<endl<<endl;
22 Compare<char> cmp3('a','A');      //定义对象cmp3，用于两个字符的比较
23 cout<<cmp3.max( )<<" is the Maximum of two characters."<<endl;
24 cout<<cmp3.min( )<<" is the Minimum of two characters."<<endl;
25 return 0;}
```



```
C:\WINDOWS\system32\cmd.exe
7 is the Maximum of two integer numbers.
3 is the Minimum of two integer numbers.

93.6 is the Maximum of two float numbers.
45.78 is the Minimum of two float numbers.

a is the Maximum of two characters.
A is the Minimum of two characters.
请按任意键继续. . .
```

需要补充说明：

(1) 类模板的类型参数可以有一个或多个，每个类型前面都必须加 `class`，如 `template<class T1, class T2>`

```
class someclass
{...};
```

在定义对象时分别代入实际的类型名，如：`someclass<int, double> obj;`

(2) 和使用类一样，使用类模板时要注意其作用域，只能在其有效作用域内用它定义对象。

(3) 模板可以有层次，一个类模板可以作为基类，派生出派生模板类。

(4) 上述介绍是在类模板中定义成员函数。如果在类模板外定义成员函数，应写成类

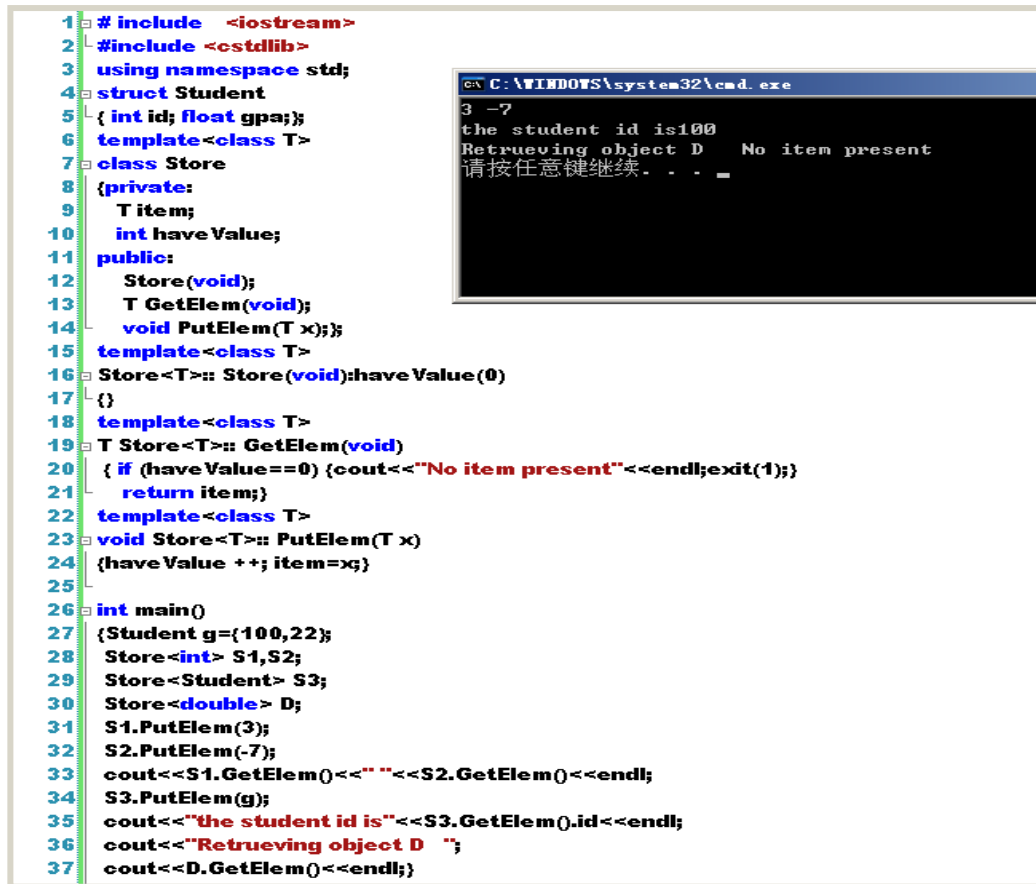
模板形式：

template<class 虚拟类型参数>

函数类型 类模板名<虚拟类型参数>::成员函数名(形参表列)

{函数体...}

下面程序实例 6-4 就是在类模板 store 之外对 3 个成员函数进行定义的示范。



The image shows a C++ program in a text editor on the left and its execution output in a command prompt window on the right. The program defines a `Student` struct and a `Store` class template. The `Store` class has private members `item` and `haveValue`, and public methods `Store()`, `GetElem()`, and `PutElem()`. The `main` function creates a `Student` object `g` and three `Store` objects: `S1` (int), `S2` (double), and `S3` (Student). It then calls `PutElem` and `GetElem` on these objects. The command prompt shows the output: `3 -7`, `the student id is 100`, `Retrueving object D No item present`, and a prompt to press any key to continue.

```
1 #include <iostream>
2 #include <cstdlib>
3 using namespace std;
4 struct Student
5 { int id; float gpa;};
6 template<class T>
7 class Store
8 {private:
9     T item;
10    int haveValue;
11 public:
12     Store(void);
13     T GetElem(void);
14     void PutElem(T x);};
15 template<class T>
16 Store<T>::Store(void):haveValue(0)
17 {}
18 template<class T>
19 T Store<T>::GetElem(void)
20 { if (haveValue==0) {cout<<"No item present"<<endl;exit(1);}
21   return item;};
22 template<class T>
23 void Store<T>::PutElem(T x)
24 {haveValue ++; item=x;};
25
26 int main()
27 {Student g={100,22};
28   Store<int> S1,S2;
29   Store<Student> S3;
30   Store<double> D;
31   S1.PutElem(3);
32   S2.PutElem(-7);
33   cout<<S1.GetElem()<<" "<<S2.GetElem()<<endl;
34   S3.PutElem(g);
35   cout<<"the student id is"<<S3.GetElem().id<<endl;
36   cout<<"Retrueving object D ";
37   cout<<D.GetElem()<<endl;}
```

```
C:\WINDOWS\system32\cmd.exe
3 -7
the student id is 100
Retrueving object D No item present
请按任意键继续. . .
```

前面介绍了类模板的申明和实例化问题。有同学可能会问：类可以继承与派生。那么类模板是否可以继承与派生呢？这个问题的答案是肯定的。在类模板继承与派生机制中，存在 2 种形式，第一种情况是用类模板来派生出新的类模板； 第 2 情况是用模板类派生派生类。

(1) 使用类模板来派生出新的类模板的基本格式

template <class Type>

class 派生类的类模板: public 基类的类模板

{派生类类模板定义};

(2) 使用模板类来派生新的类模板的基本格式

template <class Type>

class 派生类的类模板: public 基类的类模板<实参类型>

{派生类类模板定义};

下面通过一个实例程序 6-5 来说明类模板的派生规则和使用方法。

实例 6-5，类模板的派生，通过链表（Tlist）类模板派生了一个集合（Tset）类模板。

```
#include <iostream>          //包含头文件。使用iostream库
using namespace std;         //使用std命名空间
class SNode //定义链表的结点结构类
{public:
    SNode( int value );
    ~SNode( ) { };
    int m_value;             //结点值
    SNode *m_next;           //结点后继，指向下一个结点的指针
template <class Type> //定义链表的类模板
class TList
{public:
    TList( );                //构造函数
    ~TList( );               //析构函数
    virtual bool Insert( Type value ); //在链表头部插入一个结点
    bool Delete( Type value );      //从链表中删除值为value的一个结点
    bool Contain( Type value );     //判断链表中是否包含某结点
    void Print( );                 //输出链表结点的值
protected:
    SNode *m_head;    }; //设置为只有头指针的单向链表

template<class Type> //定义集合类模板——集合与链表采用同样的组织方式；
//但是链表和集合的概念不同：集合中不允许有重复的结点
class TSet : public TList<Type>
{public:
    bool Insert( Type value );}; //在集合中重载插入方法，插入前先判断结点是否已经存在
SNode::SNode( int value )//结点类构造函数
{
    m_value = value;           //结点值
    m_next = NULL;            //结点后继
template <class Type> //构造函数
TList<Type>::TList( )
{
    m_head = NULL;            //链表头

template <class Type> //链表析构函数中需要delete所有还在链表中的结点
TList<Type>::~~TList( )
{
    SNode *p = m_head;        //
    for ( ; p != NULL; )      //直到结点不为空
    {m_head = p->m_next;       //头结点指向下一个结点，作为新的头结点
        delete p;              //释放p所指向的结点
        p = m_head;            //p指向新的头结点
    }
}

template <class Type> //在链表头部插入
```



```

bool TList<Type>::Insert( Type value )
{
    SNode *pTemp = new SNode( value );    //构造一个新结点
    if ( pTemp == NULL )                  //结点空间申请不成功, 退出
    {return false;    }

    pTemp->m_next = m_head;                //新结点的m_next指针指向头结点原来所指的结点
    m_head = pTemp;                        //头结点改为新生成的结点
    return true;}

template <class Type>
bool TList<Type>::Delete( Type value )
{
    SNode *p1, *p2;                      //申请两个结点指针, 用于结点操作时要遍历整个链表以查找
    //其中是否包含有结点值为value的结点, 但是一次遍历只能够删除一个值为value结点
    //如果在链表中有多个值相同的结点, 需要分别删除。

    if ( m_head->m_value == value )        //如果头结点就是要找的结点, 直接删除
    {
        p1 = m_head->m_next;              //p1指向头结点的后继结点
        delete m_head;                    //释放头结点
        m_head = p1;                      //p1成为新的头结点, 即原头结点的后继成为新的头结点
        return true;                      //返回真
    }
    else                                  //要删除的结点非头结点
    {
        for ( p1 = m_head, p2 = m_head->m_next; p2 != NULL; )//遍历链表
        {if ( p2->m_value == value )        //如果找到, 则释放该结点, 并结束遍历
            {
                p1->m_next = p2->m_next;    //p1指向p2的下一个结点
                delete p2;                  //释放p2
                p2 = NULL;                  //让p2指向NULL, 该步骤通常很有必要
                return true;                //结束遍历。
            }
            else                            //如果该结点不是要找的结点, 则p1, p2向后遍历
            {
                p1 = p1->m_next;            //p1指向其后继
                p2 = p2->m_next;            //p2指向其后继
            }
        }

        return false;}

template <class Type>
bool TList<Type>::Contain( Type value )
{
    //遍历链表以查找其中是否包含有结点值为value的结点, 有返回true, 没有返回false
    for ( SNode *p = m_head; p != NULL; p = p->m_next )
    {
        if ( p->m_value == value )        //找到该结点, 返回true
        {return true;}
    }

    return false;}

template <class Type> //显示表中的结点数据
void TList<Type>::Print( )

```

```

{   cout << "结点的值依次为: ";    //遍历链表以读出每一个结点值并显示
    for ( SNode *p = m_head; p != NULL; p = p->m_next )
        {cout << " " << p->m_value << " ";    }
    cout << endl;}

template <class Type>
bool TSet<Type>::Insert( Type value )//集合类的结点插入方法
{   //集合中无值为value的结点，才可以作插入操作，否则直接返回false。插入失败也返回false
    if ( !( TList<Type>::Contain( value ) )&&( TList<Type>::Insert( value ) ) )
        {return true;}
    return false;}

//主测试程序
void main( )
{   TList<int> sIntList;
    sIntList.Insert( 12 );
    sIntList.Insert( 24 );    //在链表sIntList中，两次插入
    sIntList.Insert( 48 );
    sIntList.Insert( 96 );
    sIntList.Insert( 24 );    //在链表sIntList中，两次插入
    sIntList.Print( );

    sIntList.Delete( 24 );    //删除一次
    sIntList.Print( );

    TSet<int> sIntSet;
    sIntSet.Insert( 12 );
    sIntSet.Insert( 24 );    //在集合sIntList中，两次插入
    sIntSet.Insert( 48 );
    sIntSet.Insert( 96 );
    sIntSet.Insert( 24 );    //在集合sIntList中，两次插入
    sIntSet.Print( );

    sIntSet.Delete( 24 );    //删除一次
    sIntSet.Print( );

    system("pause");
}

```

程序运行结果为：

结点的值依次为： 24； 96； 48； 24； 12；

结点的值依次为： 96； 48； 24； 12；

结点的值依次为： 96； 48； 24； 12；

结点的值依次为： 96； 48； 12；

在分析上述程序时，需要注意如下问题。

- (1) 通过链表 (Tlist) 类模板派生了一个集合 (Tset) 类模板。但链表和集合是 2 种不同的类，它们的主要差别体现为：链表可以有相同数值的结点，但集合则不能。因此，这就体现在这 2 种类中数据插入函数的差异。
- (2) 为了程序简单起见，规定只在表头插入，可以删除指定值的节点，但限定一次只能够删除一个节点。
- (3) 认真体会本程序实例，该例子在后续课程《数据结构》课程可能会有很大参考价值。

6.2 异常处理

6.2.1 异常处理的概念

程序设计中，常见的错误有两大类：语法错误和运行错误。在编译时，编译系统能发现程序中的语法错误。有的程序虽然能通过编译，也能投入运行。但是，在运行过程中遇到特殊情况会出现异常，得不到正确的运行结果，甚至导致程序不正常终止，或出现死机现象。因此，程序开发者不仅要考虑程序在没有任何错误下的运行情况，更多的需要考虑程序存在某种错误时程序的运行情况。这就需要在设计程序时，应当事先分析程序运行时可能出现的各种意外的情况，并且分别制订出相应的处理方法，这就是程序的“异常处理”的机制。因此，所谓异常处理指的是对运行时出现的差错以及其他例外情况时采用特别指定方式来进行处理。

下面列举了在程序设计中应该考虑的一些例外情况，但不限于这些情况。

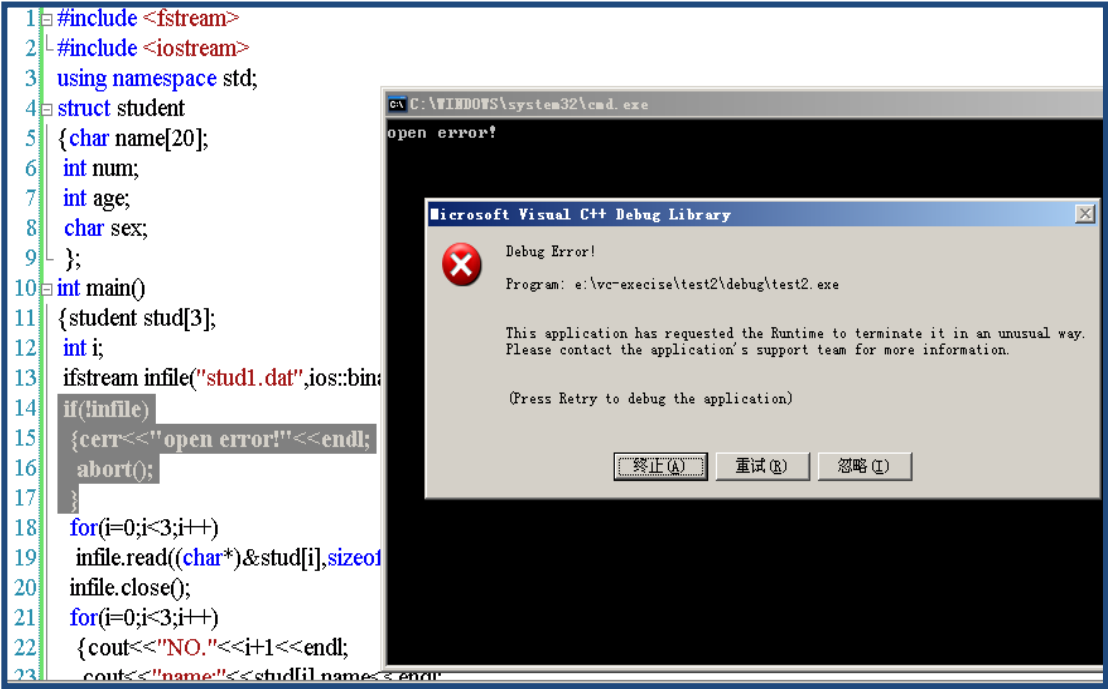
- (1) 文件打开失败；
- (2) 内存分配失败；
- (3) 外部函数模板调用失败；
- (4) 非法指针；
- (5) 非法运算（如除数为零）；
- (6) 数组访问越界；
- (7) 函数输入、输出参数超出预期范围；
- (8) 未初始化的变量使用；
- (9) 一算法逻辑错误等等。

在运行没有异常处理的程序时，如果运行情况出现异常，由于程序本身不能处理，程序只能终止运行，这时可能会出现用户事先未料想到的结果，甚至严重的错误。如果在程序中设置了异常处理机制，则在运行情况出现异常时，由于程序本身已规定了处理的方法，于是程序的流程就转到异常处理代码段处理。用户可以指定进行任何的处理，这样就可以保证程序能在正常或异常情况下，都能按照用户事先确定的程序过程来执行，保证了程序运行的稳定性。

6.2.2 异常处理的方法

在一些小规模程序中，可以用比较简单的方法处理异常。例如，在程序设计时，通过分析采用 if 语句，对一些可能出现例外的语句进行判断（通常通过函数返回值来判断）：在语句正常情况下程序如何执行；在非正常情况下，程序又该执行哪些代码？下面通过一个程序实例来说明一般简单情况下的例外处理方法。

实例 6-6 简单程序的例外处理方法。



在上述程序实例 6-6 中，第 14-17 行语句就是对文件打开可能失败的例外处理。通过分析可以发现，例外处理程序代码包括了“例外发现”和“例外处理”2 类语句。例如，第 14 条主要功能是发现例外，第 15-17 条语句主要功能是例外处理。通过这 2 类语句的协同，就可以保证程序在正常或异常情况下都能正常稳定地运行。因此，传统的例外处理基本过程如图 6-1 所示。

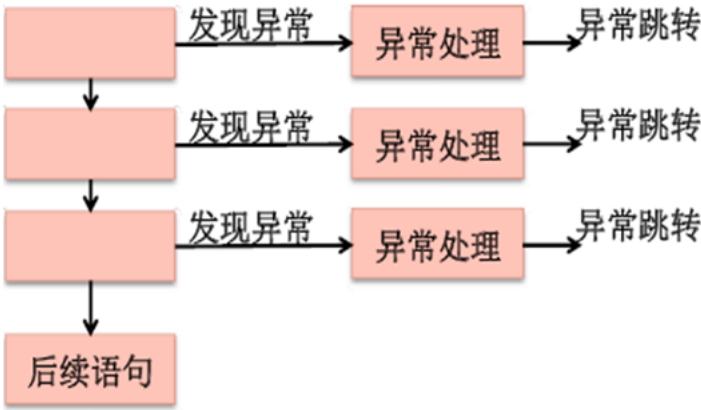


图6-1 传统的例外处理基本过程

但是在一些规模比较大的程序系统中，如果在每一个函数中都设置处理异常的程序段，会使程序过于复杂和庞大。因此，前面介绍的传统异常处理方法虽然解决了程序意外崩溃问题，却带来了代码混乱问题。基于规模生产中的分工和协作的基本思想，大型程序设计中，引入了结构化的异常处理机制，该机制对例外处理的规范做法是将“异常发现”和“异常处理”环节分离。在 C++ 中，这种结构化的异常处理机制体现在 2 个方面。

(1) 发现与处理分离机制。使底层的函数专门用于解决实际任务，而不必再承担处理异常的任务，以减轻底层函数的负担，而把处理异常的任务上移到某一层去处理，可提高效率。

(2) 逐级上报机制。如果在执行一个函数过程中出现异常，发出一个信息给它的上一级（即调用它的函数），上级捕捉到信息后进行处理。如果上一级的函数也不能处理，就再传给其上一级。如此逐级上报。如果到最高一级还无法处理，最后系统调用 `Terminate()` 终止程序。

这样做使异常的发现与处理不由同一函数来完成。好处是使底层的函数专门用于解决实际任务，只负责定义和发现例外，而不必再承担处理异常的任务，以减轻底层函数的负担，而把处理异常的任务上移到某一层去处理。这样可以提高效率。

根据上述异常处理的基本思想。C++ 处理异常的机制是引入了 3 类语句来规范例外处理程序的设计。这 3 类语句分别是异常检查 (`try`)、异常定义与抛出 (`throw`)，以及异常捕捉与处理 (`catch`) 等。把需要检查的语句放在 `try` 块中，`throw` 用来当出现异常时发出一个异常信息（数据类型），而 `catch` 则用来捕捉异常信息，如果捕捉到了异常信息，就处理它。基本过程如图 6-2 所示。因此，C++ 异常处理的一般格式如下。

1) `throw` 语句一般形式：`throw 表达式`；在（下级）函数中上报异常类型

2) `try-catch` 的结构：

`try`

 {被检查的语句}

`catch(异常信息类型 [变量名])`

 {进行异常处理的语句}

在（上级）函数中捕获（下级）上报的错误类型

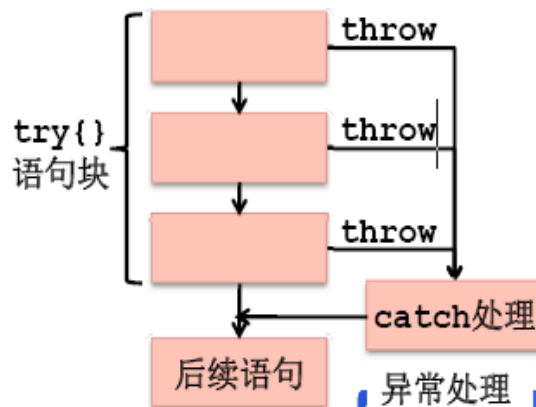


图6-2 C++异常处理基本过程

下面通过程序实例来说明上述语句的使用方法。

实例 6-7. 给出三角形的三边 a, b, c , 求三角形的面积。只有 $a+b>c, b+c>a, c+a>b$ 时才能构成三角形。设置异常处理, 对不符合三角形条件的输出警告信息, 不予计算。在函数 `tria` 中对三角形条件进行检查, 如果不符合三角形条件, 就抛出一个异常信息, 在主函数中的 `try-catch` 块中调用 `tria` 函数, 检测有无异常信息, 并作相应处理。相应的程序代码如下。

```

#include <iostream>
#include <cmath>
using namespace std;
void main( )
{
    double tria(double, double, double);
    double a1, a2, a3;
    cin>>a1>>a2>>a3;
    try          //在try块中包含要检查的函数
    {
        while(a1>0 && a2>0 && a3>0)
        {
            cout<<tria(a1, a2, a3)<<endl;
            cin>>a1>>a2>>a3;
        }
    }
    catch(double)      //用catch捕捉异常信息并作相应处理
    {
        cout<<"a1="<<a1<<", a2="<<a2<<", a3="<<a3<<", that is not a triangle!"<<endl;
    }
    cout<<"end"<<endl;
}

double tria(double a, double b, double c) //计算三角形的面积的函数
{
    double s=(a+b+c)/2;

```

```

if (a+b<=c || b+c<=a || c+a<=b) throw a; //当不符合三角形条件抛出异常信息
return sqrt(s*(s-a)*(s-b)*(s-c));
}

```

程序运行结果如下：

```

6 5 4          (输入 a, b, c 的值)
9.92157        (计算出三角形的面积)
1 1.5 2        (输入 a, b, c 的值)
0.726184       (计算出三角形的面积)
1 2 1          (输入 a, b, c 的值)
a=1,b=2,c=1, that is not a triangle! (异常处理)
end

```

下面分析上述程序实例异常处理过程。首先把可能出现异常的、需要检查的语句或程序段放在 try 后面的花括号中。程序开始运行后，按正常的顺序执行到 try 块，开始执行 try 块中花括号内的语句。如果在执行 try 块内的语句过程中没有发生异常，则 catch 子句不起作用，流程转到 catch 子句后面的语句继续执行。如果在执行 try 块内的语句(包括其所调用的函数)过程中发生异常，则 throw 运算符抛出一个异常信息(数据类型)。throw 抛出异常信息后，流程立即离开本函数，转到其上一级的函数(main 函数)。在进行异常处理后，程序并不会自动终止，继续执行 catch 子句后面的语句。

因此，结合程序实例，可以总结出 C++异常处理的一些基本规则。

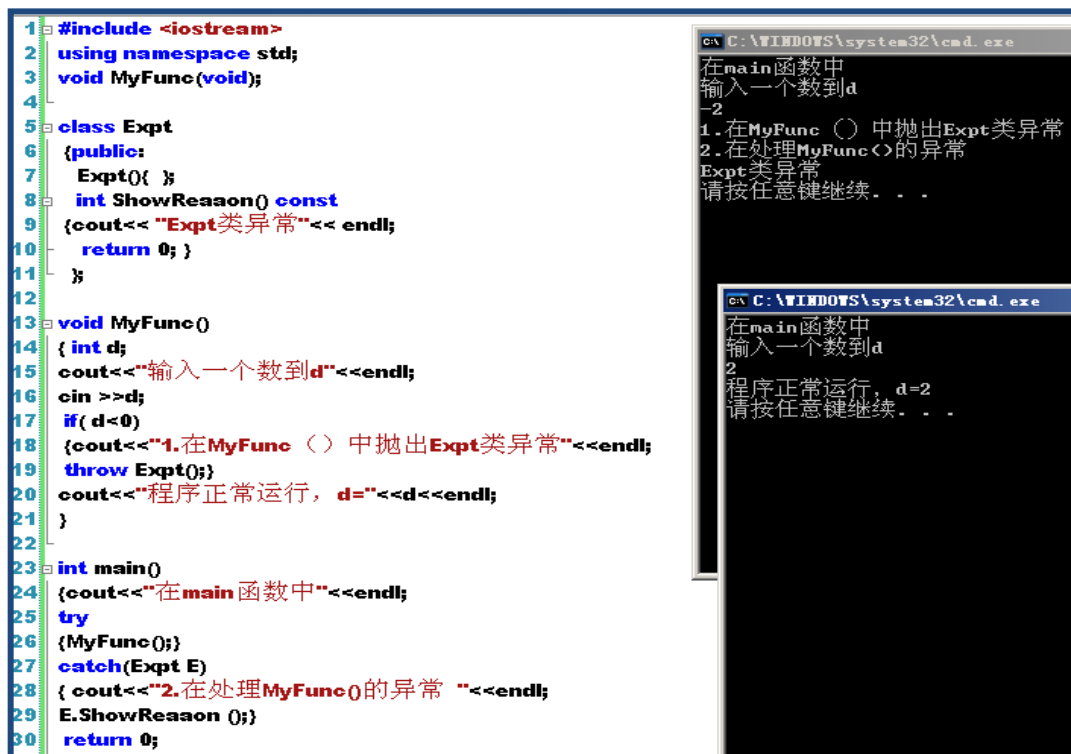
- (1) try 和 catch 作为整体出现，catch 块必须紧跟在 try 块后。在二者之间也不能插入其他语句。但在一个 try-catch 结构中，可只有 try 块而无 catch 块、或多个 catch 块。且两者都得用花括号括起来；
- (2) catch 括号中，只写异常信息的类型名，如 catch(double)。catch 只检查所捕获异常信息的类型，而不检查它们的值。如没有指定类型，而用了删节号“...”，则表示可捕捉任何类型的异常信息；
- (3) try-catch 结构可与 throw 出现在同函数中，也可不在同一函数中。当 throw 抛出异常信息后，首先在本函数中寻找与之匹配的 catch，如在本函数中无 try-catch 结构或找不到与之匹配的 catch，就转到最近上级 try-catch 结构去处理；
- (4) throw 可不包括表达式，表示“不处理此异常，请上级处理”；如果某段程序发现了自己不能处理的异常，就可以使用 throw 表达式抛出这个异常，将它抛给调用者。
- (5) 如果 throw 抛出的异常信息找不到与之匹配的 catch 块，那么系统就会调用一个系统函数 terminate，使程序终止运行。
- (6) catch 在数据类型匹配时，可以是标准类型或用户自定义类型(如类或类对象)。在进行类型匹配时，catch 出现的顺序很重要。因为在一个 try 块中引发异常时，异常处理程序按照它在 catch 中出现的顺序进行检查。catch 的排列顺序

应按照从特殊到一般的排列顺序（指异常对象从特殊到一般排列）。异常匹配不要求在异常定义和异常处理器之间匹配得十分完美。一个对象或一个派生类对象的引用可以与基类处理器匹配。所以，异常处理 catch 语句的排列顺序应该将派生类对象的异常捕获放在前面，而将基类异常对象的捕获放在处理程序后面。

为了进一步深入理解上述规则，下面设计一个简单的嵌套调用结构的异常程序处理实例来加以说明。

下面通过程序实例 6-8 来加以说明。

实例 6-8 异常信息为用户自定义类型的异常处理方法。



```
1 #include <iostream>
2 using namespace std;
3 void MyFunc(void);
4
5 class Expt
6 {public:
7     Expt(){};
8     int ShowReason() const
9     {cout<<"Expt类异常"<< endl;
10      return 0; }
11 };
12
13 void MyFunc()
14 { int d;
15  cout<<"输入一个数到d"<<endl;
16  cin >>d;
17  if( d<0)
18  {cout<<"1.在MyFunc () 中抛出Expt类异常"<<endl;
19   throw Expt();}
20  cout<<"程序正常运行, d="<<d<<endl;
21 }
22
23 int main()
24 {cout<<"在main函数中"<<endl;
25 try
26 {MyFunc();}
27 catch (Expt E)
28 { cout<<"2.在处理MyFunc()的异常 "<<endl;
29  E.ShowReason ();}
30 return 0;
```

```
C:\WINDOWS\system32\cmd.exe
在main函数中
输入一个数到d
-2
1.在MyFunc () 中抛出Expt类异常
2.在处理MyFunc()的异常
Expt类异常
请按任意键继续. . .

C:\WINDOWS\system32\cmd.exe
在main函数中
输入一个数到d
2
程序正常运行, d=2
请按任意键继续. . .
```

在上述程序中，MyFunc() 函数的异常程序代码段 throw 出一个 Expt 类的异常类型，因此，在 catch() 语句中捕获的也是 Expt 类的对象 E，两者相互匹配。

在使用类或类对象来定义异常信息时，需要特别关注：为了恢复程序的正确执行，需要保证异常抛出时对象被正确地清除。C++的异常处理器可以保证离开作用域时，作用域中的所有结构完整的对象的析构函数都被调用，以清除这些对象。下面通过程序实例 6-9 来说明异常处理过程中的对象构建与析构问题

实例 6-9 异常处理过程中的对象构建与析构

```
#include <iostream>
using namespace std;
void MyFunc(void);
class CMyException
{
public:
```



```

    CMyException() {} ;
    ~CMyException() {} ;
    const char *ShowExceptionReson() const
    {
        return "Exception in CMyException class.";
    }
};

class CDoctorDemo
{
public:
    CDoctorDemo() ;
    ~CDoctorDemo() ;
};

CDoctorDemo::CDoctorDemo()
{
    cout << "Constructing CDoctorDemo." << endl;
}

CDoctorDemo::~CDoctorDemo()
{
    cout << "Destructing CDoctorDemo." << endl;
}

void MyFunc()
{
    CDoctorDemo D;
    cout << "In MyFunc(). Throwing CMyException exception." << endl;
    throw CMyException();
}

int main()
{
    cout << "In main." << endl;
    try
    {
        cout << "In try block, calling MyFunc()." << endl;
        MyFunc();
    }
    catch( CMyException E )
    {
        cout << "In catch handler." << endl;
        cout << "Caught CMyException exception type: ";
    }
}

```

```

        cout << E.ShowExceptionReson() << endl;
    }
    catch( char *str )
    {
        cout << "Caught some other exception: " << str << endl;
    }
    cout << "Back in main. Execution resumes here." << endl;
    return 0;
}

```

程序运行结果为:

```

In main.
In try block, calling MyFunc( ).
Constructing CDoctorDemo.
In MyFunc( ). Throwing CMyException exception.
Destructing CDoctorDemo.
In catch handler.
Caught CMyException exception type: Exception in CMyException class.
Back in main. Execution resumes here.

```

通过程序实例 6-9 分析可知,如果在 try 块(或 try 块中调用的函数)中定义了类对象,在建立该对象时要调用构造函数。在执行 try 块(包括在 try 块中调用其他函数)的过程中如果发生了异常,此时流程立即离开 try 块。这样流程就有可能离开该对象的作用域而转到其他函数。因而,应当事先做好结束对象前的清理工作,C++的异常处理机制会在 throw 抛出异常信息被 catch 捕获时,对有关的局部对象进行析构(调用类对象的析构函数),析构对象的顺序与构造的顺序相反,然后执行与异常信息匹配的 catch 块中的语句。

6.2.3 函数声明中的异常情况指定

前面介绍了一个函数在申明时,需要指明函数类型、函数名称以及参数和参数类型等信息。如果结合本章学习异常处理机制,C++函数的完整申明还需要指明该函数是否有异常处理情况。因此,异常规格申明是 C++函数申明的一部分,它们指定了函数可以抛出什么异常。这样使得用户在看程序时能够知道所用的函数是否会抛出异常信息以及异常信息可能的类型。例如

```

void f1() throw(int); //可以抛出一个整型异常;
void f2() throw(char *, E); //可以抛出一个 char *或一个 E (这里 E 是用户自定义类型)类型的异常。
void f3() throw(); //表明函数不抛出异常。

```

在 throw 无参数时,则该声明表示一个不能抛出异常的函数,这时即使在函数执行过程中出现了 throw 语句,实际上也并不执行 throw 语句,并不抛出任何异常信息,程序将非正常终止。

```
void f4();//表明函数可以抛出任何类型异常信息。
```

因为如果在声明函数时未列出可能抛出的异常类型,则该函数可以抛出任何类型的异常信息。

需要注意:异常指定是函数声明的一部分,必须同时出现在函数声明和函数定义的首行中,否则在进行函数的另一次声明时,编译系统会报告“类型不匹配”。

6.3 命名空间

在学习本书前面各章时,我们在程序设计时都需要写上一条以下的语句, `using namespace std;`。这是为什么呢?本节将揭开谜底的答案。

6.3.1 命名空间概念和作用

命名空间是 ANSI C++引入的另一个编程工具。命名空间可以理解为一个可以由用户命名的作用域,用来解决程序设计中常见的同名冲突。在 C++中,有不同类型的作用域,如:文件(编译单元)作用域、函数和复合语句作用域、类作用域等。在不同的作用域中可以定义相同名字的变量,互不干扰。编译系统可以通过作用域空间名字对不同作用域的同名标识进行区别。在前面介绍过函数和复合语句的作用域,下面只对类作用域和文件作用域进行分析,加深对作用域和命名空间的作用的理解。

```
class A          //声明A类
{
public:
    void f1( );   //声明A类中的f1函数
private:
    int data1;
};
void A::f1( )    //定义A类中的f1函数
{
    //
}
class B          //声明B类
{
public:
    void f1( );   //B类中也有f1函数
    void f2( );
private:
    int data1;
};
void B::f1( )    //定义B类中的f1函数
{
    //
```

```
}
```

在上述程序中，尽管在类 A 和 B 中都出现了成员函数 f1()和数据成员 data1；但它们属于不同的类作用域。这样不会发生混淆。f1()函数在进行引用时，只需要在函数前面加上类名和限定符就可以确定唯一标识。例如 void B::f1()；表示是 B 类中的 f1 函数。同样，B::data1 和 A::data1 是不同的两个变量。

另外，在大型程序设计时，一个程序往往由多人分别编写，这样就决定了一个程序由多个文件组成。由于各文件（或头文件）是由不同的人设计的，有可能在不同的文件（或头文件）中用了相同的名字来命名所定义的类或函数。这些具有相同名称的变量或函数在分开编译时是不会出错的。因为它们隶属于不同的文件作用域。但是如果使用#include 命令行将这些头文件包含进来，编译连接时，就会错误。也就说这些同名的变量或函数在整个程序作用域中就会出现名字冲突。下面通过程序实例 6-10 加以说明。

实例 6-10 文件作用域及程序中同名冲突问题示例。

程序员甲在头文件 header1.h 中定义了类 S 和函数 f()

```
#include <iostream>
#include <string>
#include <cmath>
using namespace std;
class S //声明S类
{
public:
    S(int n,string nam,char s)
    {
        num=n;name=nam;sex=s;
    }
    void get_data( );
private:
    int num;
    string name;
    char sex;
};
void S::get_data() //成员函数定义
{
    cout<<num<<" "<<name<<" "<<sex<<endl;
}
double f(double a,double b) //定义全局函数(即外部函数)
{
    return sqrt(a+b);
}
```

如果程序员乙写了头文件 header2.h，在其中除了定义其他类以外，还定义了类 S 和函

数 f ()，但其内容与头文件 header1.h 中的 S 和函数 f () 有所不同。

```
#include <iostream>
#include <string>
#include <cmath>
using namespace std;
class S    //声明S类
{
public:
    S(int n,string nam,char s)  //参数与header1中的s不同
    {
        num=n;name=nam;sex=s;
    }
    void get_data( );
private:
    int num;
    string name;
    char sex;                //此项与header1不同
};
void S::get_data( )          //成员函数定义
{
    cout<<num<<" "<<name<<" "<<sex<<endl;
}
double f(double a,double b)  //定义全局函数
{
    return sqrt(a-b);
}                               //返回值与header1中的f () 函数不同
//头文件中可能还有其他内容
```

假如主程序员在其程序中要用到 header1.h 中的 S 和函数 f()，因而在程序中包含了头文件 header1.h，同时要用到头文件 header2.h 中的一些内容，因而在程序中又包含了头文件 header2.h。如果主文件(包含主函数的文件)如下。

```
#include <iostream>
#include "header1.h"  //包含头文件
#include "header2.h"  //包含头文件
using namespace std;
int main( )
{
    S stud1(101,"Wang",18);
    stud1.get_data();
    cout<<f(5,3)<<endl;
    return 0;
}
```

这时程序编译就会出错。因为在预编译后，头文件中的内容取代了对应的#include 命

令行，这样就在同一个程序文件中出现了两个 S 类和两个 f 函数，显然是重复定义，这就是名字冲突，即在同一个程序作用域中有两个或多个同名的实体。

不仅如此，在程序中还往往需要引用一些库，为此需要包含有关的头文件。如果在这些库中包含有与程序的全局实体同名的实体，或者不同的库中有相同的实体名，则在编译时就会出现名字冲突。

6.3.2 同名冲突解决方法与命名空间

1. 同名冲突解决方法

通过分析上述不同作用域的命名问题，我们能感知到在大型程序多人合作编写时，很容易出现在整个程序空间中的同名冲突问题。为了避免这类问题的出现，人们提出了许多方法，例如：将实体的名字写得长一些；把名字起得特殊一些，包括一些特殊的字符；由编译系统提供的内部全局标识符都用下划线作为前缀，如 `_complex()`，以避免与用户命名的实体同名；由软件开发商提供的实体的名字用特定的字符作为前缀。但是这样的效果并不理想，而且增加了阅读程序的难度。为此，在新版 ANSI C++ 标准中，通过引入命名空间(namespace) 机制来解决该问题。命名空间机制能够将程序的全局实体与其他库的全局标识符区别开来。

所谓命名空间，实际上就是一个由程序设计者命名的内存区域。程序设计者可以根据需要指定一些有名字的空间域，把一些全局实体分别放在这些又名字的空间域中，实现与其他全局实体的分隔。

例如，在某程序中，通过引入一个命名的内存空间 `ns`，将程序中使用的全局变量 `a, b` 保存在该内存空间中。程序代码如下：

```
namespace ns //指定命名空间 ns
{
    int a;
    double b;
}
```

现在命名空间成员包括变量 `a` 和 `b`，注意 `a` 和 `b` 仍然是全局变量，仅仅是把它们隐藏在指定的命名空间中而已。

如果在程序中要使用变量 `a` 和 `b`，必须加上命名空间名和作用域分辨符“`::`”，如 `ns1::a`，`ns1::b`。这种用法称为命名空间限定(qualified)法。这些名字（如 `ns1::a`）称为被限定名(qualified name)。如果一个程序中的全局变量，分别被分配在不同的命名空间中，通过命名空间限定法就可以解决程序域的同名冲突问题。因此，命名空间机制主要目的参加会议在于解决名字（用户定义的类型名、变量名和函数名等标识）冲突的问题。

需要特别注意：命名空间和变量一样，也必须遵循先定义、后使用的原则。下面分别介绍命名空间定义和使用方法。

2. 命名空间的申明（定义）方法

如何在一个程序中命名一个内存空间呢？C++ 提供了一个命名空间的语句 `namespace`。

该语句使用一般格式如下。

```
namespace 空间名
{变量(可以带有初始化);
    常量;
    函数(可以是定义或声明);
    结构体;
    类;
    模板;
    命名空间(命名空间中又定义命名空间, 即嵌套)
}
```

例如, 下面就是在某程序中申明了一个命名空间 `ns`, 并在该空间中定义了一些全局变量、函数等。特别地, 在该空间中, 还嵌套地申明了第 2 层的命名空间 `ns`, 并在 `nss` 中定义了变量 `age`。

```
namespace ns
{
    const int R=0.08;//常量
    int a;
    double p;                //变量
    double t()                //函数
    {
        return a*R;
    }
    namespace nss             //嵌套的命名空间
    {
        int a;
    }
}
```

3. 命名空间中成员的使用方法

在命名空间外使用命名空间有两种方法: 一种是使用作用域运算符“`::`”; 另外一种就是使用 `using` 关键字。

(1) 命名空间限定法

如果想输出命名空间 `ns1` 中成员的数据, 可以采用命名空间限定法, 唯一地来表示这些全局标识。例如在上例中命名空间 `ns` 的基础上, 如果要使用其中标识, 可以标识为:

```
cout<<ns::R<<endl;
cout<<ns::p<<endl;
cout<<ns::a<<endl;    //需要注意嵌套空间中变量 a 的表示区别
cout<<ns::t()<<endl;
cout<<ns::nss::a<<endl; //需要指定外层的和内层的命名空间名
```

下面通过一个完整的程序实例 6-11 来说明命名空间的作用和使用方法。

实例 6-11 利用命名空间来解决实例 6-10 程序名字冲突问题。修改两个头文件，把在头文件中声明的类分别放在两个不同的命名空间中。

//header1.h (头文件)

#include <string>

#include <cmath>

using namespace std;

namespace ns1 //声明命名空间ns1

{

class S //在命名空间ns1内声明S 类

{

public:

S (int n, string nam, int a)

{

num=n;name=nam;age=a;

}

void get_data();

private:

int num;

string name;

int age;

};

void S::get_data() //定义成员函数

{

cout<<num<<" "<<name<<" "<<age<<endl;

}

double f(double a, double b) //在命名空间ns1内定义f () 函数

{

return sqrt(a+b);

}

}

//header2.h ((头文件)

#include <string>

#include <cmath>

using namespace std;

namespace ns2 //声明命名空间ns2

{

class S

{

public:

S (int n, string nam, char s)

{

num=n;name=nam;sex=s;

}


```

        void get_data( );
private:
    int num;
    string name;
    char sex;
};
void S::get_data( )
{
    cout<<num<<" "<<name<<" "<<sex<<endl;
}
double f(double a,double b)
{
    return sqrt(a-b);
}
}

//main file (主文件)
#include <iostream>
#include "/header1.h"           //包含头文件
#include "/header2.h"           //包含头文件
using namespace std;
int main( )
{
    ns1::S stud1(101,"Wang",18); //用命名空间ns1中声明的Student类定义stud1
    stud1.get_data( );           //不要写成ns1::stud1.get_data( );
    cout<<ns1::f(5,3)<<endl;      //调用命名空间ns1中的fun函数
    ns2::S stud2(102,"Li",'f');  //用命名空间ns2中声明的Student类定义stud2
    stud2.get_data( );
    cout<<ns2::f(5,3)<<endl;      //调用命名空间ns1中的fun函数
    return 0;
}

```

程序能顺利通过编译，并得到以下运行结果：

```

101 Wang 18(对象 stud1 中的数据)
2.82843           (5+3 的开方值)
102 Li f         (对象 stud2 中的数据)
1.41421          (5-3 的开方值)

```

(2) using 命名空间:: 成员名

使用 using 后面可以接命名空间成员名。例如

```

using ns1::R;
using ns1::T();

```

当有了上述申明后, `cout<<T( )<<endl;` //T()函数相当于 `ns1:: T()`;

注意: `using` 声明的有效范围是从 `using` 语句开始到 `using` 所在的作用域结束。另外, 在同一作用域中用 `using` 声明命名空间的成员时, 如果有同名的成员则无法这样使用。

(3) `using namespace` 命名空间名

该方法能用一个语句就能一次声明一个命名空间中的全部成员。

例如: `using namespace ns;`

`cout<<T()<<endl;` //此处的 T() 函数相当于 `ns:: T()`;

表示在本作用域中要用到命名空间 `ns1` 中所有成员时都不必用命名空间限定。

现在请大家思考一下, 为什么我们在刚开始学习 C++ 程序设计时, 需要在程序的开头都要加上一个语句: `using namespace std;`

答案: 为了解决 C++ 标准库中的标识符与程序中的全局标识符之间以及不同库中的标识符之间的同名冲突, 应该将不同库的标识符在不同的命名空间中定义(或声明)。标准 C++ 库的所有的标识符都是在一个名为 `std` 的命名空间中定义的, 或者说标准头文件(如 `iostream`) 中函数、类、对象和类模板是在命名空间 `std` 中定义的。这样, 在程序中用到 C++ 标准库时, 需要使用 `std` 作为限定。如 `std::cout<<" OK. " <<endl;`

在大多数的 C++ 程序中常用 `using namespace` 语句对命名空间 `std` 进行声明, 这样可以不必对每个命名空间成员一一进行处理, 所以, 在文件的开头加入一套语句: `using namespace std;` 声明。

当然, 也可以使用若干个“`using` 命名空间成员”声明来代替“`using namespace std;`”声明, 如

```
using std::string;
using std::cout;
using std::cin;等。
```

为了减少在每一个程序中都要重复书写以上的 `using` 声明, 程序开发者往往把编写应用程序时经常会用到的命名空间 `std` 成员的 `using` 声明组成一个头文件, 然后在程序中包含此头文件即可。

为了更全面地体会命名空间的定义和使用方法, 下面列举程序实例 6-12 来加以说明。

程序实例 6-12 命名空间定义和使用。

```
#include <iostream>
using namespace std;
int Num = 1;
namespace Name1
{
    int Num = 2;
    int Add(int Num)
    {
```

```

        Num = ::Num+Num;  //::Num 表示全局作用域
        return Num;
    }
}

namespace Name2
{
    int Num = 3;
    int Add(int Num)
    {
        Num = ::Num + Num;
        return Num;
    }
}

void main(void)
{
    cout << Name1::Add(4)+::Num << endl;
    cout <<Name2::Add(5)+Num << endl;
    namespace N2 = Name1;
    cout << N2::Add(6)+N2::Num << endl;
    using namespace Name2;
    cout << Add(7)+Name2::Num << endl;
}

```

程序运行结果为

```

6
7
9
11
请按任意键继续. . .

```

6.4 C++的函数库

C 语言程序中各种功能基本上都是由函数来实现的，在 C 语言的发展过程中建立了功能丰富的函数库，C++从 C 语言继承了这份宝贵的财富。在 C++程序中可以使用 C 语言的函数库。如果要用 C 语言函数库中的函数，就必须在程序文件中包含有关的头文件，在不同的头文件中，包含了不同的函数的声明。这些头文件名包括后缀.h，如 stdio.h，math.h 等。由于 C 语言没有命名空间，头文件并不存放在命名空间中，因此，在 C++程序文件中如果用到带后缀.h 的头文件时，不必用命名空间。只需在文件中包含所用的头文件即可。标准 C 语言的函数库主要有图 6-3 所示的种类。

```

#include <assert.h>           //设定插入点
#include <ctype.h>             //字符处理
#include <errno.h>             //定义错误码
#include <float.h>             //浮点数处理
#include <fstream.h>           //文件输入 / 输出
#include <iomanip.h>            //参数化输入 / 输出
#include <iostream.h>          //数据流输入 / 输出
#include <limits.h>            //定义各种数据类型最值常量
#include <locale.h>            //定义本地化函数
#include <math.h>              //定义数学函数
#include <stdio.h>             //定义输入 / 输出函数
#include <stdlib.h>            //定义杂项函数及内存分配函数
#include <string.h>            //字符串处理
#include <strstream.h>         //基于数组的输入 / 输出
#include <time.h>              //定义关于时间的函数
#include <wchar.h>             //宽字符处理及输入 / 输出
#include <wctype.h>           //宽字符分类

```

图6-3 C语言中函数头文件

另外，标准 C++ 语言也还提供了一套函数库（能覆盖 C 语言的函数库所有功能）。这些 C++ 标准要求系统提供的头文件不包括后缀 .h。例如 `iostream`、`string`。为了表示与 C 语言的头文件有联系又有区别，C++ 所用的头文件名是在 C 语言的相应的头文件名（但不包括后缀 .h）之前加一字母 `c`。此外，由于这些函数都是在命名空间 `std` 中声明的，因此，在程序中要对命名空间 `std` 作声明。图 6-4 列举了 C++ 提供的主要函数头文件。

标准 C++ （同上的不再注释）

```
#include <algorithm>      //STL 通用算法
#include <bitset>          //STL 位集容器
#include <cctype>
#include <cerrno>
#include <clocale>
#include <cmath>
#include <complex>        //复数类
#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <ctime>
#include <deque>          //STL 双端队列容器
#include <exception>      //异常处理类
#include <fstream>
#include <functional>     //STL 定义运算函数（代替运算符）
#include <limits>
#include <list>            //STL 线性列表容器
#include <map>             //STL 映射容器
#include <iomanip>
#include <ios>             //基本输入 / 输出支持
#include <iosfwd>          //输入 / 输出系统使用的前置声明
#include <iostream>
#include <istream>         //基本输入流
#include <ostream>        //基本输出流
#include <queue>           //STL 队列容器
#include <set>             //STL 集合容器
#include <sstream>         //基于字符串的流
#include <stack>           //STL 堆栈容器
#include <stdexcept>       //标准异常类
#include <streambuf>       //底层输入 / 输出支持

#include <string>          //字符串类
#include <utility>         //STL 通用模板类
#include <vector>          //STL 动态数组容器
#include <cwchar>
#include <cwctype>
```

图6-4 C++的标准函数库头文件

至于上述头文件中申明的具体函数，可以通过 C++ 编译工具的联机帮助或者上网就可以查询。由于内容太多，在此就不一一列举了。另外，需要注意的是，目前所用的大多数 C++ 编译系统既保留了 C 的用法，又提供了 C++ 的新方法。例如，下面两种用法等价，可以任选。

C 传统方法

```
#include <string.h>
```

C++ 新方法

```
#include <cstring>
```

```
using namespace std;
```

6.5 C++标准模板库

前面学习了 C++ 提供的函数模板和类模板工具, 通过这些工具的使用, 在很多程度上改进了 C++ 的代码重用性。为了进一步提高 C++ 的代码重用性, 美国加州的惠普实验室的 Alex Stepanov, Meng Lee, David R Musser 3 位科学家在认识到 C++ 程序员经常使用许多数据结构和算法后, 研发了标准模板库 (Standard Template Library, STL)。1998 年 C++ 标准委员会在 C++ 的标准函数库中增加了 STL 内容。STL 提供了强大的功能, 它是基于模板、可重用组件, 实现了许多通用数据结构以及用于处理这些数据的算法。STL 虽然是一套程序库文件, 但不是一般概念上的程序库, 而是一个有着划时代意义, 背后拥有着先进技术与深厚理论的工具, 可以说它是软件重用技术发展史上一项重大突破。

长久以来, 软件界一直希望建立一种可重复运用的东西, 以及一种可以制造出“可重复运用的东西”的方法, 让软件工程师或程序员的心血不至于随时间的推移、人事异动而烟消云散。从程序 (procedures)、函数 (functions)、类别 (classes), 到函数库 (function libraries)、类别库 (class libraries)、各种组件 (components), 从结构化设计、模块化设计、再到样式 (patterns) 的归纳整理, 无一不是软件工程的漫漫奋斗史, 为的就是代码重用性 (reuseability) 的提升。

STL 就是在这个背景下诞生的。STL 的价值在两方面。低层次而言, STL 带给我们一套极具价值的零组件, 这种价值就像 MFC 对于 Windows 开发过程所带来的价值一样, 直接而明朗。高层次而言, STL 还带给我们一个以泛型编程 (generic programming) 思维为基础的程序设计理念。所谓泛型程序设计是指编写不依赖于具体数据类型的程序, 将算法从特定的数据结构中抽象出来成为通用的, 也称为通用程序设计。

6.5.1 STL 的组件以及关系

STL 提供 4 大组件, 彼此可以组合套用。

1. 容器 (containers): 各种数据结构。例如, Vector、List、Map 等, 用来存储各种数据。
2. 算法 (algorithms): 各种常用算法。例如, sort、search、copy 等, 这些算法的作用是为程序提供各种常用的操作。
3. 迭代器 (iterators): 一个非常重要的组件, 用来将容器和算法联系起来。也就是通常所说的泛型指针。
4. 函数对象 (function object): 行为类似函数, 可作为演算法的某种策略 (policy)。

STL 4 大组件的关系如下图 6-5 所示。

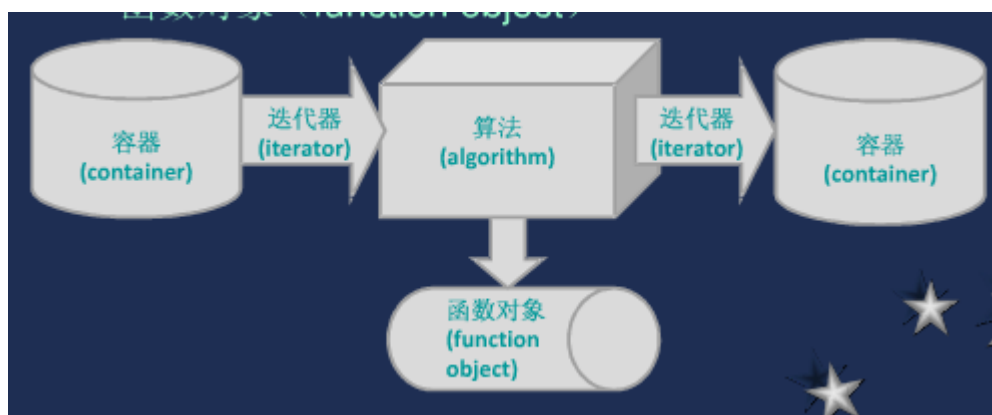


图 6-5 STL 的组件及其关系

下面对 STL 中常用的组件做一些概念上的介绍，详细内容涉及到的相关技术比较多，在此难以深入介绍。因此，本节只能对 STL 工具的学习起着抛砖引玉的作用。

6.5.2 容器

1. 容器概念及类型

容器可以理解为常用的数据结构的模板化，用来表示各种数据结构对象或者可以理解主要用于存放数据，每个容器表现为类模版。容器分为 3 类，序列容器类、关联容器和容器适配类。

(1) 序列容器也称为顺序容器。它表示线性数据结构，以线性方式存储序列元素，并且这些序列元素有头有尾，依次存放。序列的“头”是序列的首元素。序列的“尾”是序列的末元素。对于这些元素的访问，总可以从首元素出发，逐个访问每个中间元素，然后到达最后一个元素。向量(vector)、链表(list)、双端队列(deque)都是典型的顺序容器。根据顺序容器的特点，可采用两种方式进行访问。

i. 顺序访问:顺序访问必须从首元素开始逐渐递增到目标元素，向量 vector 和双端队列 deque 也可以使用该种方式，而链表 list 只能使用该方式访问内部的元素。

ii. 随机访问:随机访问类似于对传统数组的访问，如对于向量 vector 和双端队列 deque，给定下标就可以直接找到对应的元素的引用。

(2) 关联容器表示非线性数据结构。它通常用来快速找到容器中存储的元素。这种容器可以存储值的集合或者键/值对。这类容器主要有集合(set)、多重集合(multiset)、映射(map)和多重映射(multimap)。

关联容器中的元素没有严格线性关系，所以其中的元素没有首元素和末元素的区别。对于关联容器中的元素一般采用索引方式进行访问。

(3) 容器适配器是以某种受限的方式对顺序容器进行访问。例如，堆栈（访问方式 LIFO）和队列（访问方式 FIFO）就是一种受限访问的顺序容器。因此，容器适配器有特定的访问接口和方法。

2. 容器对象的构造和析构

采用上述容器模板类，就可以构建容器对象来存储不同类型的数据结构。所有容器类都提供了不带参数的默认构造函数，可用这种方式构造出空的容器对象，然后向容器内插入元素。所有容器类还提供了构造函数利用一个给定的数据区间来构造容器。另外，所有容器类还提供了一个拷贝构造函数能以现有容器对象为样本复制生成内容相同的另一个容器。

在容器对象使用完成之后，还需要对容器对象进行析构，因此，所有容器都提供了一个析构函数来释放容器元素所占用的存储空间。表 6-1 所示列举了 C++容器对象的构造和析构。

表 6-1 C++容器对象的构造和析构

语句	说明
<code>Containers<T> c;</code>	定义指定类型T的容器对象，利用默认构造函数
<code>Containers<T> c(begin,begin+N);</code>	利用一个给定的数据区间[<code>beg</code> , <code>beg+N</code>) 来构造容器对象。注意：包含 <code>beg</code> 位置元素，不包含 <code>beg+N</code> 位置的元素
<code>Containers<T> c(c1);</code>	拷贝构造对象
<code>~Containers()</code>	析构函数释放元素所占内存
<code>c=c1;</code>	对象复制
<code>c.swap(c1);</code>	实现两个容器对象内容的互换
其中Containers 是容器类，T为指定的数据类型，c、c1为容器对象	

3. 容器对象的维护操作

在构建容器对象之后，就可以对所有容器对象进行如下维护操作。

插入元素：向某容器对象中插入一个或多个元素；

删除元素：删除某容器对象中一个或多个元素；

清空容器：删除某容器对象中所有元素。

另外，还可以查询容器的容量和大小，判断是否为空等。有关容器对象维护操作的相关方法如表 6-2 所示。

表 6-2 容器对象维护操作

函数	说明
c.insert(pos,e)	在指定位置插入数据元素e
c.erase(pos)	删除指定位置的元素
c.erase(begin,e nd)	删除区间[begin, end) 中的元素
c.clear()	删除容器内所有的元素
c.max_size()	返回容器对象的容量
c.size()	然会容器对象当前所包含元素的数量
c.empty()	返回容器是否为空
c为容器对象，pos、begin、end均为迭代器	

下面通过一个程序实例，结合向量容器来说明容器的构造与操作方法。向量 vector 是 STL 提供的最简单、最常用的顺序容器模板类，用于容纳不定长的线性序列，允许对各元素进行随机访问，这一点与 C++ 数组操作基本相同。但是向量 vector 的大小是灵活可变的，可以看作是一个动态数组，在运行时可以自由改变自身的大小以便容纳任何数目的元素。向量 vector 提供了对序列元素的快速、随机访问。由于其本身的结构与设计特点，在其末端的插入和删除元素速度最快，效率最高。当然在序列中其他位置插入、删除也是完全可以的，但是这样效率会降低，因为 vector 对象必须要移动元素位置来容纳新的元素或者收回被删除元素的空间。

前面的内容已经介绍了顺序容器的主要共性，向量 vector 除了支持这些共性以外，还有其本身的特征：新增构造函数 vector(size_type n, const T&value=T()); 该函数用来初始化一个指定大小 n 的向量对象；另外，对于随机元素访问，vector 类重载了 “[]” 运算符，允许使用下标直接访问序列元素。

实例 6-13 容器的构造与操作示例

```
#include <iostream>
#include <vector>
using namespace std;
void print(vector<float> &vct)    //输出向量的每一个元素
{
    size_t numelem=vct.size();
    for(size_t i=0;i<numelem;i++)
        cout<<vct[i]<<" ";
    cout<<endl;
}
int main()
```

```

{
    vector<float> v1, v2; //定义了两个空类型为float的向量容器对象
    v1.push_back(3.14f); //插入数据到向量v1中（末端）
    v1.push_back(21.7f);
    for(size_t i=0; i<3; i++)
        v2.push_back(2.6f*(i+1)); //循环实现插入3个数据到向量v2中
    cout<<"v1: ";
    print(v1); //输出v1内容
    cout<<"v2: ";
    print(v2); //输出v2内容
    cout<<"v2 after erase: ";
    v2.erase(v2.begin()+1); //删除v2中下标为1的元素（5.2）
    print(v2); //输出v2内容
    v1.swap(v2); //实现两个向量容器的内容互换
    cout<<"v1: ";
    print(v1); //输出v1内容
    cout<<"v2: ";
    print(v2); //输出v2内容
    return 0;
}

```

程序运行结果为：

```

v1: 3.14 21.7
v2: 2.6 5.2 7.8
v2 after erase: 2.6 7.8
v1: 2.6 7.8
v2: 3.14 21.7

```

6.5.3 迭代器

迭代器是 STL 体系中一个非常重要的概念，也是一个很抽象的概念。迭代器是算法和容器间的“桥梁纽带”，用来保存它所操作的特定容器的状态信息；它可以指向容器中的一个位置，用户不必关心这个位置的真正物理地址，无须关心其存储形式，只要通过迭代器就能访问这个位置的元素。迭代器和指针有着许多共同的性质，主要体现为：

（1）迭代器就是用面向对象技术封装的高级指针。它提供了灵活的访问形式，可以对不同的数据类型、不同的存储形式进行常用的访问操作。

（2）可以使用运算符函数“++”或“--”，让迭代器前后移动，然后用“*迭代器”的形式来访问迭代器所指向的数据。

C++定义了 5 种类型的 STL 迭代器，分别阐述如下。

（1）输入迭代器（Input iterator）：这种迭代器的层次较低，可以用来从序列中读取

数据，但是不一定能够向其中写入数据。

(2) 输出迭代器 (Output Iterator): 与输入迭代器类似，层次较低，允许向序列中写入数据，但是不一定能从其中读取数据。

(3) 前向迭代器 (Forward iterator): 既是输入迭代器又是输出迭代器，因此它既支持数据读取，也支持数据写入，并且可以对序列进行单向的遍历。

(4) 双向迭代器 (Bidirectional iterator): 功能与前向迭代器相似，区别在于双向迭代器在两个方向上都可以对数据遍历，如链表容器 list 的迭代器就符合此种类型的基本特征。

(5) 随机访问迭代器 (Random access iterator): 也称为双向迭代器，它对迭代器提出了更高的要求，能够在序列中的任意两个位置之间进行跳转，例如，向量 vector 中的迭代器就是此种类型。

另外，STL 还为迭代器提供了 3 个辅助函数 (模板)。分别简述如下。

(1) advance() 函数。该函数可以改变迭代器的位置，具体改变的幅度和方向由参数决定，本质上是在函数内部对迭代器进行了若干次迭代，然后指向新的元素，其函数模板原型为: template <typename _InIt, typename _Diff>

```
void advance(_InIt &Where, _Diff offset
```

(2) distance() 函数。该函数可以计算两个迭代器之间的距离，其函数模板原型为:

```
template <typename _InIt>
```

```
int distance(_InIt &from, _InIt &to);
```

(3) iter_swap() 函数。该函数可以交换两个迭代器所指向的元素值，而且两个迭代器的可以不必都指向同一个容器，但是要求定义的模板参数类型要相同，其函数模板原型为:

```
template <typename FwdIt1, typename FwdIt2>
```

```
void iter_swap(FwdIt1 &fi1, FwdIt2 &fi2);
```

下面结合链表容器来说明迭代器的操作方法。链表容器是一种典型的顺序容器类。其内部数据结构实质是一个双向链表，可以在任何一端操作，与向量 vector 不同的是，链表容器必须进行顺序访问，不能实现随机访问即不支持 “[]” 运算符，只能用对应迭代器操作元素。同样，除了前面的内容介绍的顺序容器的共性外，链表还有自身的特殊操作，表 6-3 列出了链表容器主要的操作。

表 6-3 链表容器主要的操作

函数调用	说明
l.push_front()	把数据插入到链表对象l的首端
l.splice(pos,a)	把链表对象a中的元素插入到当前链表中pos之前，并清空链表a
l.splice(pos, a, posa)	把链表a中从位置posa后的元素转移到l 的位置pos之前
l.splice(pos, a, abeg, aend)	把链表a中在区间[abeg, aend) 内的元素转移到l 的位置pos之前
l.unique()	删除链表中相邻的重复的元素
l.remove(x)	删除与x相等的元素
l.sort()	对链表排序
l.reverse()	逆转链表中元素的次序

实例 6-14 链表容器对象的迭代器的操作方法示例

```
#include <list>
using namespace std;
int main()
{
    int ary[10]={1, 7, 9, 3, 2, 8, 6, 5, 4, 0};
    list<int> l1(ary, ary+10); //定义链表，数据用数组ary初始化
    list<int>::iterator iter;    //定义一个迭代器
    iter=l1.begin();            //迭代器指向l1首元素
    advance(iter, 5);           //向后移动5个元素
    cout<<*iter<<" ";         //输出
    iter=l1.end();              //迭代器指向l1末端
    advance(iter, -2);          //向前移动2个元素
    cout<<*iter<<endl;         //输出
    return 0;
}
```

运行结果

8 4

为了进一步理解容器和迭代器的概念，下面将结合顺序容器中另一常用的容器，即双队列容器来加以说明。双队列(deque)与向量容器(vector)相似，deque 也是一种动态数组的形式，是一种访问形式比较自由的双端队列，可以从队列的两端入队及出队（添加和删除），也可以使用运算符“[]”通过给定下标形式来访问队列中的元素，既可以顺序访问，也提供了随机访问的能力。由于本身内部结构的特点，在队列两端添加和删除元素时，速度最快，效率较高，而在中间插入数据时比较费时，因为必须移动其他元素来实现容器的扩张。deque 容器的常见操作如表 6-4 所示。

表 6-4 deque 容器的常见操作

函数调用	说明
d.push_front()	把数据插入到deque对象d的首端
d.pop_front()	删除首端元素，无返回值
d.push_back()	把数据插入到deque对象d的末端
d.pop_back()	删除末端元素，无返回值
d.operator[] (index)	使用运算符“[]”访问容器中的对象，index为给定下标

实例 6-15 双端队列容器 deque 的使用

```

#include <iostream>
#include <iostream>
#include <deque>
using namespace std;
void print(deque < double > &deq) //输出队列中的每一个元素
{
    size_t numelem= deq.size(); //队列内元素的数量
    for(size_t i=0;i<numelem;i++) cout<< deq[i]<<" ";
    cout<<endl;
}
void main()
{
    deque<double> ds; //定义了空的类型为double的deque对象
    for(size_t i=0;i<5;i++) //循环再首端插入5个数据，并输出队列内容
    {
        ds.push_front(1.2*i); //插入数据
        print(ds);
    }
    ds.pop_back(); //删除末端元素
    print(ds); //输出队列内容
}

```

运行结果

```

0
1.2 0

```

```
2.4  1.2  0
3.6  2.4  1.2  0
4.8  3.6  2.4  1.2  0
4.8  3.6  2.4  1.2
```

通过上述程序实例，可以归纳出容器类迭代器的基本操作具有如下特点：

一般编程最常用迭代器就是容器类迭代器，用容器提供的迭代器类型定义变量一般遵循如下格式：容器类名称<数据类型实参>::iterator 迭代器对象；

其中容器类可以是 list、vector、deque、set、map 等，数据类型实参可以是任何数据类型，但是定义迭代器绝不是孤立的，其数据类型实参必须与迭代器要指向的容器元素类型相同。

STL 中的容器有很多种，不同容器定义的迭代器也不一样，但都提供一组相同的函数接口，以便访问迭代器所指位置中的数据，并驱动迭代器在有效区间上移动。

STL 容器所提供两种迭代器：Container<T>::iterator 类型的迭代器允许读、写元素；Container<T>::const_iterator 类型定义的迭代器访问元素是只读的。

另外，将迭代器操作接口和函数可以总结为表 6-5 所示。

表 6-5 迭代器操作接口和函数

操作和函数调用	说明
Container<T>::iterator it	定义容纳T类型数据容器的迭代器的方法
Container<T>::const_iterator	定义容纳T类型数据容器的const迭代器的方法
c.begin()	返回容器对象c指向首元素的迭代器
c.end()	返回容器对象c指向末元素后一个位置的迭代器（不是末元素的位置）
it= c.begin ()	迭代器赋值
* it	返回迭代器所指位置中的数据，类似指针操作
++, --操作	前移/后移迭代器所指位置，包括it++,++it,it--,--it
==	判断迭代器所指位置是否相同
Container是容器类，c为容器对象，it为迭代器对象	

实例 6-16 容器的迭代器常用操作

```
#include <iostream>
#include <vector>
using namespace std;
int main()
```

```

{
    int a[5]={1, 3, 5, 7, 9};
    vector<int> v1(a, a+5);           //整型数向量容器
    vector<double> v2;                //浮点数向量容器
    vector<int>::iterator it;          //普通迭代器
    vector<int>::const_iterator cit;   //只读迭代器
    v2.push_back(3.14);
    //it=v2.begin();    此语句出错，容器元素类型和迭代器指向类型不一致
    for (it=v1.begin(); it!=v1.end(); it++)
        (*it)+=20;           //循环使v1中每个元素加10
    for (cit=v1.begin(); cit!=v1.end(); cit++)
        cout <<*cit <<" ";   //利用只读迭代器输出元素值
    cout <<endl;
    return 0;
}

```

运行结果

21 23 25 27 29

6.5.4 算法与函数对象

算法和函数对象是 STL 中另外 2 个重要的概念，而且这 2 个概念密切相关。因此，下面在本节中将重点介绍这 2 个概念。

1. 算法

算法在 STL 中体现为一系列的函数模板。这些函数模板是通用的，可适用于不同类型的数据。STL 算法的操作对象以序列中（如图 6-6 所示）的元素为主，并以迭代器作为函数参数，这些迭代器参数必须指向容器中的元素，并且构成一个数据集合的区间。因此，算法可以灵活地处理不同长度的数据集合。

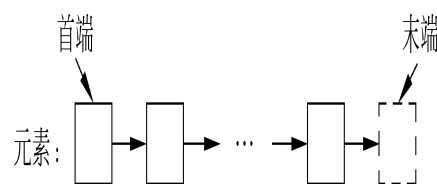


图 6-6 算法操作的对象

C++标准模板库中包括 70 多种算法，其中包括查找算法、排序算法、消除算法、记数算法、比较算法、变换算法、置换算法和容器管理等。大致可以分成四大类：

（1）不可变序列的算法。这类算法在对容器进行操作时不会改变容器的内容，这类算法比较典型的有查找 `find()`、计数 `count()`、比较 `equal()` 等。

（2）可变序列的算法。这些算法执行完毕后，序列中的元素的数值和数量会发生变化，如复制 `copy()`、反转 `reverse()`、填充 `fill()` 等；

(3) 排序相关的算法。这类算法主要的特点是对序列的内容进行不同方式的排序，包括合并算法、二分查找算法以及有序列的集合操作算法等，典型的是 `sort()` 函数；

(4) 通用数值算法。这类算法主要是对序列内容进行数值计算，如累加 `accumulate()`，邻接与求差 `adjacent_difference()`、求绝对值等。

2. 算法使用的基本方法

在 C++ 类的封装中，一般类库做法是将算法作为类行为嵌入在容器类中。STL 将算法从容器中分离出来，使得算法的扩展更加容易，也使得 STL 的效率更高，避免了调用虚拟函数的开销。在 STL 中，算法最终是以函数模板的形式来实现的。因此，对算法的使用本质就是对函数模板的调用。另外，分析前面介绍的迭代器相关知识可以知道，所有的迭代器都会提供最基本的共性操作，那就是迭代器可以指向序列中的元素，迭代器对象本身可以递推指向下一个元素。因此，任何算法的使用在语法上都必须满足最低层次迭代器的要求，也就是说，在算法的函数模板调用时，无论函数参数和函数返回值（算法结果）都是采用迭代器来表示。下面结合一个程序实例来说明这一点。

实例 6-17 数据拷贝算法（copy 函数）的使用实例

```
#include <iostream>
#include <vector>
#include <iterator>
using namespace std;
int main()
{
    int a[5]={1, 3, 5, 7, 9};
    vector<int> v1(a, a+5);           //整型数向量, 利用数组初始化
    ostream_iterator<int> output(cout, " "); //定义输出流迭代器
    copy(v1.begin(), v1.end(), output); //把v1内容复制给输出流迭代器
    return 0;
}
```

运行结果

1 3 5 7 9

分析上述程序可知，程序中是通过使用拷贝算法（copy 函数）来实现向量容器对象之间的复制。拷贝算法的 `copy()` 函数模板的原型如下：

```
template <typename InputIterator, typename OutputIterator >
OutputIterator copy(InputIterator beg, InputIterator end, OutputIterator
output);
```

在以上的函数原型中隐含着如下的信息：

(1) 其中定义 `InputIterator` 抽象类型代表输入型迭代器，`OutputIterator` 代表输出型迭代器，满足了最低层次的迭代器的要求；

(2) 操作对象定义在一个序列区间内，由 `InputIterator` 定义的 `beg` 和 `end` 就代表了这样的操作区间 `[beg, end)`；

(3) copy() 函数的返回值也是 OutputIterator 迭代器类型。

3. 函数对象

在 STL 的算法中,有些算法函数可以使用默认的规则。而对于某些特殊的要求,则需要给算法函数一个特殊的规则,而这种特殊的规则就需要用函数对象来实现。因此说,函数对象和算法是紧密相关的一组概念。

那么如何理解函数对象呢?所谓“函数对象”其实就是一个行为类似于函数的对象。它可带也可不带参数,其功能就是获得一个值,或者是改变算法操作的状态。因此,定义函数对象就是定义了一个特殊的类,在该类中重载了具有 public 访问权限的运算符“()”,这在语法和语义上相对于使用了 operator() 的函数。这个类的对象就可称为函数对象。在 C++ 中,普通函数和函数对象都可以作为算法的参数。

除了可以通过自定义函数对象来实现算法的不同操作形式外,STL 中也定义了一些标准的函数对象。按功能可以划分为:算术运算、关系运算、逻辑运算 3 类。有关这些标准函数对象的定义是在头文件(functional)中。具体情况可以查阅相关资料。

实例 6-18 采用默认规则和函数对象定义的特殊规则,通过使用 sort 算法函数的进行排序。

```
#include <algorithm>
#include <iostream>
#include <vector>
#include <iterator>
#include <functional>
using namespace std;
int main()
{
    int ary[7]={1, 7, 3, 4, 6, 5, 9};
    vector<int> v1(ary, ary+7);           //整型数向量
    ostream_iterator<int> output(cout, " "); //输出流迭代器
    cout<<"Original data: ";
    copy(v1.begin(), v1.end(), output);
    cout<<endl<<"Sort ascending: ";
    sort(v1.begin(), v1.end());           //用默认规则排序(升序)
    copy(v1.begin(), v1.end(), output);
    cout<<endl<<"Sort descending: ";
    sort(v1.begin(), v1.end(), greater<int>()); //给定降序规则
    copy(v1.begin(), v1.end(), output);
    cout<<endl;
    return 0;
}
```

运行结果

Original data: 1 7 3 4 6 5 9

Sort ascending: 1 3 4 5 6 7 9

Sort descending: 9 7 6 5 4 3 1

在 C++ 中，普通函数和函数对象都可以作为算法的参数。在实例 6-19 中就使用普通函数作为算法 `for_each` 的参数，把序列中的元素逐个传给 `power` 函数，并调用该算法函数。在程序实例 6-20 中，使用类的对象作为函数对象，在类定义中重载函数调用运算符，通过函数对象来调用排序函数，实现学生对象的排序。

实例 6-19 采用普通函数作为算法参数调用排序算法实现排序功能。

```
#include <iostream>
#include <algorithm>
#include <vector>
using namespace std;
int power(int x)      //定义power函数，实现求平方并输出值
{
    int pr=x*x;
    cout<<pr<<" ";
    return pr;
}
int main()
{
    int ary[5]={1, 3, 5, 2, 7};
    vector<int> v1(ary, ary+5);    //构造向量v1
    for_each(v1.begin(), v1.end(), power);    //调用算法，逐次执行power
    return 0;
}
```

运行结果

1 9 25 4 49

实际上，一般来说，用户设计的普通函数就可以看做是一种最简单的函数对象。例如，本程序实例中的 `power()` 函数就是一种普通函数。采用 `power()` 函数作为函数参数传递给算法 `for_each()`，指导 `for_each()` 算法对向量容器对象 `v1` 中元素的排序规则。

实例 6-20 采用函数对象对学生对象的排序。

```
#include <iostream>
#include <algorithm>
#include <vector>
#include <string>
using namespace std;
class Student
{
public:
    int number;
    string name;
    Student(int i, string s)
    {
        number=i;
        name=s;
    }
}
```

```

    }
    void Print()
    {
        cout<<number<<" "<<name<<endl;
    }
};

class numbercmp      //定义numbercmp类
{
public:
    bool operator() (Student &st1, Student &st2)    //重载运算符“()”
    {
        return st1.number<st2.number;
    }          //比较学号
};

int main()
{
    //定义三个学生对象，学号没有按升序设置
    Student st1(1003, "wang"), st2(1001, "li"), st3(1002, "zhao");
    vector<Student> v1;    //定义向量v1
    v1.push_back(st1);    //把st1, st2, st3顺序插入向量中
    v1.push_back(st2);
    v1.push_back(st3);
    sort(v1.begin(), v1.end(), numbercmp()); //使用函数对象numbercmp()作为规则进行排序
    for(int i=0; i<3; i++) v1[i].Print(); //输出每个学生信息（已经按升序排好）
    return 0;
}

```

运行结果

```

1001, li
1002, zhao
1003, wang

```

在该程序中，通过类 numbercmp 来重载运算符 operator()，就定义了一种可以作为函数参数的对象，同样可以像使用程序实例 6-19 中的普通函数 power() 一样来使用该对象。但是，此时传递给算法 sort() 的对象是通过 numbercmp 类的默认构造函数来获得的。使用类的形式定义的函数对象比普通函数携带更多的额外信息。

6.6 综合程序应用——犀利人事管理系统

在前面一章的综合程序应用的基础上，对“犀利人事管理系统”进行功能扩展和修改。要求如下：增加一个文件打开或者读写异常处理类，实现文件访问的异常处理功能，即当文件打开或读写不成功时，程序将抛出异常信息，由主程序通过屏幕告诉用户错误原因。

FileName:database.h

Copyright: Tsinghua University

Author: Huaizhou Tao

Date:2014-10-02

Description: 程序中作为数据库所使用的类的声明

```

*****/
#ifndef DATABASE_H
#define DATABASE_H

#include <string>
#include "linkedlist.h"
#include "employee.h"
#include "manager.h"
#include "technician.h"
#include "salesman.h"
#include "salemanager.h"
#include "fileexception.h"
using std::string;

enum EmployeeType
{
    Employee,
    Manager,
    Technician,
    Salesman,
    SaleManager
};

//数据库类，对程序使用的数据结构进行封装
class database
{
private:
    //数据成员为不同种类雇员的对象链表
    linkedlist<manager> mngList;
    linkedlist<technician> techList;
    linkedlist<salesman> saleList;
    linkedlist<salemanager> salemngList;
public:
    //构造函数与析构函数
    database();
    ~database();

    //文件读写函数

```

```

void load();
void save();

void loadManager(const string &fileName);
void loadTechnician(const string &fileName);
void loadSalesman(const string &fileName);
void loadSaleManager(const string &fileName);

void saveManager(const string &fileName);
void saveTechnician(const string &fileName);
void saveSalesman(const string &fileName);
void saveSaleManager(const string &fileName);

//删除雇员函数
bool deleteEmployee(const string &name);
bool deleteEmployee(int empNo);

//添加雇员函数
//由于每类雇员信息不同，各有不同的添加函数
void addManager();
void addTechnician();
void addSalesman();
void addSaleManager();
//添加雇员对外接口
void addEmployee(enum EmployeeType);

bool updateEmployee(int empNo);

//显示详细信息
void detailInfo(int empNo);
//显示所有雇员
int showAll(enum EmployeeType);

//查找函数，可按姓名与编号查找
int search(int empNo);
int search(const string &name);

//排序函数，可按编号或工资排序
void sortByPay(enum EmployeeType, int direction);
void sortByNo(enum EmployeeType, int direction);

void sortCustom(employee **head, int length, int direction, int keycol);

int compare(employee *e1, employee *e2, int direction, int keycol);

```

```

        //设置销售经理与销售员的关系
        bool setRelation(int salesmanNo, int saleManagerNo);
};

#endif

*****
FileName:employee.h
Copyright: Tsinghua University
Author: Huaizhou Tao
Date:2014-10-02
Description: employee类的声明

*****/
#ifndef EMPLOYEE_H
#define EMPLOYEE_H

#include <string>
using std::string;

//employee类, 对公司的雇员信息进行封装
class employee
{
private:
    //三个私有数据成员, 分别代表雇员的编号(individualEmpNo)、等级(grade)与月薪(accumPay)
    int individualEmpNo;
    int grade;
    int accumPay;
    //该成员代表雇员的姓名
    string name;
    //这个静态私有成员用于在新加入雇员时使编号自动增加
    static int currentEmpNo;
public:
    //无参构造函数与有参构造函数
    employee();
    employee(int inputEmpNo, int inputGrade, int inputPay, string inputName);

    //析构函数
    ~employee();

    //访问数据成员的接口函数, 分别是
    //设置编号: setEmpNo(int)

```

```

//设置等级: setGrade(int)
//设置月薪: setAccumPay(int)
//获取编号: int getEmpNo()
//获取等级: int getGrade()
//获取月薪: int getAccumPay()
//由于派生类的工资基于另外的规则进行设置, 故不建议直接使用setAccumPay接口
void setEmpNo(int inputEmpNo);
void setGrade(int inputGrade) {grade = inputGrade;};
void setAccumPay(int inputPay) {accumPay = inputPay;};
int getEmpNo() {return individualEmpNo;};
int getGrade() {return grade;};
int getAccumPay() {return accumPay;};

//关于姓名字符串的接口
void setName(string inputName) {name = inputName;};
string getName() {return name;};

//该函数的功能是打印雇员的基本信息
virtual void printInfo();
virtual void updateInfo();

//新的成员函数, 用于计算工资与调级
virtual int PAY() {return accumPay;};
virtual bool promote(int gradeUp);

//流输入运算符重载, 从流向对象输入基本信息
friend std::istream& operator >>(std::istream&, employee&);

//流输出运算符重载, 向流输出对象的基本信息
friend std::ostream& operator <<(std::ostream&, employee&);
};

#endif

/*****
FileName:fileexception.h
Copyright: Tsinghua University
Author: Huaizhou Tao
Date:2014-10-20
Description: 文件异常类的声明

*****/
#ifndef FILEEXCEPTION_H
#define FILEEXCEPTION_H

```

```

class FileException
{
public:
    string filename;
    string type;
    string mode;

    FileException(string inputFilename, string inputType, string inputMode)
    {
        filename = inputFilename;
        type = inputType;
        mode = inputMode;
    };

    ~FileException() {};
};

#endif

/*****
FileName:linkedlist.h
Copyright: Tsinghua University
Author: Huaizhou Tao
Date:2014-10-02
Description: linkedlist类的声明

*****/
#ifndef LINKEDLIST_H
#define LINKEDLIST_H

//基于模板的链表结点定义
template <class T>
struct node
{
    T data;
    node<T> *next;
};

//基于模板的链表类声明
template <class T>
class linkedlist
{

```



```

private:
    node<T> *head;          //头结点
    node<T> *current;       //当前结点
    //内联函数，用于深拷贝
    inline void deepCopy(const linkedlist<T> &original);
public:
    //构造函数，拷贝构造函数，析构函数
    linkedlist();
    linkedlist(const linkedlist<T> &aplist);
    ~linkedlist();

    void insert(node<T> *newNode);          //在头部之前插入元素
    void insert_end(node<T> *newNode);      //在尾部插入
    node<T>* getFirst();                    //获得链表头的数据
    inline node<T>* getNext();              //获得当前结点的下一个数据
    bool find(const T &element);            //查找一个数据
    bool retrieve(T &element);              //检索一个数据
    bool replace(const T &newElement);      //替换一个数据
    bool remove(node<T> *node);             //移除一个数据
    bool isEmpty() const;
    void makeEmpty();
    int size();
    //重载"="运算符
    linkedlist<T>& operator =(const linkedlist<T> &rlist);
};

```

```

#endif

```

```

*****

```

FileName:manager.h

Copyright: Tsinghua University

Author: Huaizhou Tao

Date:2014-10-02

Description: manager类的声明

```

*****/

```

```

#ifndef MANAGER_H

```

```

#define MANAGER_H

```

```

#define FIXED_PAY_MANAGER 12000

```

```

#define PROMOTE_PAY_MANAGER 4000

```

```

#include "employee.h"

```

```

//manager类，继承employee类，代表经理岗位
class manager: virtual public employee
{
private:
    //增加一个成员，表示该经理的固定工资，用于调级操作
    int fixed_pay;
public:
    //无参构造函数与有参构造函数
    manager();
    manager(int inputEmpNo, int inputGrade, string inputName);

    //析构函数
    ~manager();

    void setFixedPay(int inputPay){fixed_pay = inputPay;};
    int getFixedPay(){return fixed_pay;}

    //该函数的功能是打印雇员的基本信息
    virtual void printInfo();
    virtual void updateInfo();

    //新的成员函数，用于计算工资与调级
    virtual int PAY();
    virtual bool promote(int gradeUp);

    //流输入运算符重载，从流向对象输入基本信息
    friend std::istream& operator >>(std::istream&, manager&);

    //流输出运算符重载，向流输出对象的基本信息
    friend std::ostream& operator <<(std::ostream&, manager&);
};

#endif

/*****
FileName:salemanager.h
Copyright: Tsinghua University
Author: Huaizhou Tao
Date:2014-10-02
Description: salemanager类的声明

*****/
#ifndef SALEMANAGER_H
#define SALEMANAGER_H

```

```

#define FIXED_PAY_SALEMANAGER 8000
#define RATE_SALEMANAGER 0.04
#define PROMOTE_PAY_SALEMANAGER 2000
#define PROMOTE_RATE_SALEMANAGER 0.005

#include "manager.h"
#include "salesman.h"
#include "linkedlist.cpp"

//salemanager类，继承manager类与salesman类，代表销售经理
class salemanager: public manager, public salesman
{
private:
    //使用链表保存经理手下管理的销售员的编号
    linkedlist<int> salesmanNoList;
public:
    //无参构造函数与有参构造函数
    salemanager();
    salemanager(int inputEmpNo, int inputGrade, string inputName, int inputSales,
linkedlist<int> &inputNoList);

    //析构函数
    ~salemanager();

    //成员函数：用于改变经理的销售总额
    void changeSales(int inputSales);

    //成员函数：用于添加或删除经理的下属销售员
    bool addSalesman(int empNo);
    bool deleteSalesman(int empNo);
    void clearSalesman();

    //该函数的功能是打印雇员的基本信息
    void printInfo();
    void updateInfo();

    //新的成员函数，用于计算工资与调级
    int PAY();
    bool promote(int gradeUp);

    //流输入运算符重载，从流向对象输入基本信息
    friend std::istream& operator >>(std::istream&, salemanager&);

```

```

        //流输出运算符重载，向流输出对象的基本信息
        friend std::ostream& operator <<(std::ostream&, salemanager&);
};

#endif

/*****
FileName:saleman.h
Copyright: Tsinghua University
Author: Huaizhou Tao
Date:2014-10-02
Description: salesman类的声明

*****/
#ifndef SALESMAN_H
#define SALESMAN_H

#define RATE_SALESMAN 0.05
#define PROMOTE_RATE_SALESMAN 0.01

#include "employee.h"

class salemanager;

//salesman类，继承employee类，代表销售岗位
class salesman: virtual public employee
{
private:
    //新增的私有数据成员表示销售员的月销售额与提成率
    int sales;
    double commissionRate;

    //该成员表示管理该销售员的销售经理
    salemanager *boss;
public:
    //无参构造函数与有参构造函数
    salesman();
    salesman(int inputEmpNo, int inputGrade, string inputName, int inputSales,
salemanager *inputBoss);

    //析构函数
    ~salesman();

    //访问数据成员的接口函数，分别是

```

```

//设置提成比率: setCommissionRate(double)
//设置销售额:   setSales(int)
//设置销售经理: setBoss(salemanager*)
//获取提成比率: double getCommissionRate()
//获取销售额:   int getSales()
//获取销售经理: salemanager* getBoss()
void setCommissionRate(double inputRate){commissionRate =
inputRate;setAccumPay((int)(sales * commissionRate));};
void setSales(int inputSales);
void setBoss(salemanager *inputBoss);
double getCommissionRate(){return commissionRate;};
int getSales(){return sales;};
salemanager* getBoss(){return boss;};

//该函数的功能是打印雇员的基本信息
virtual void printInfo();
virtual void updateInfo();

//新的成员函数，用于计算工资与调级
virtual int PAY();
virtual bool promote(int gradeUp);

//流输入运算符重载，从流向对象输入基本信息
friend std::istream& operator >>(std::istream&, salesman&);

//流输出运算符重载，向流输出对象的基本信息
friend std::ostream& operator <<(std::ostream&, salesman&);
};

#endif

/*****
FileName:technician.h
Copyright: Tsinghua University
Author: Huaizhou Tao
Date:2014-10-02
Description: technician类的声明

*****/
#ifndef TECHNICIAN_H
#define TECHNICIAN_H

#define WAGE_TECHNICIAN 260
#define PROMOTE_WAGE_TECHNICIAN 20

```

```

#include "employee.h"

//technician类，继承employee类，代表技术岗位
class technician: public employee
{
private:
    //新增的私有数据成员表示技术岗雇员的月工作时间与时薪
    int workHour;
    int wage;
public:
    //无参构造函数与有参构造函数
    technician();
    technician(int inputEmpNo, int inputGrade, string inputName, int inputWorkHour);

    //析构函数
    ~technician();

    //访问数据成员的接口函数，分别是
    //设置基本时薪: setWage(int)
    //设置工作时间: setWorkHour(int)
    //获取基本时薪: int getWage()
    //获取工作时间: int getWorkHour()
    void setWage(int inputWage){wage = inputWage;setAccumPay(workHour * wage);};
    void setWorkHour(int inputWorkHour){workHour =
inputWorkHour;setAccumPay(workHour * wage);};
    int getWage(){return wage;};
    int getWorkHour(){return workHour;};

    //该函数的功能是打印雇员的基本信息
    void printInfo();
    void updateInfo();

    //新的成员函数，用于计算工资与调级
    int PAY();
    bool promote(int gradeUp);

    //流输入运算符重载，从流向对象输入基本信息
    friend std::istream& operator >>(std::istream&, technician&);

    //流输出运算符重载，向流输出对象的基本信息
    friend std::ostream& operator <<(std::ostream&, technician&);
};

```

```

        #endif

/*****
FileName:userinterface.h
Copyright: Tsinghua University
Author: Huaizhou Tao
Date:2014-10-02
Description: 程序内封装用户界面所使用的类的声明

*****/
#ifndef USERINTERFACE_H
#define USERINTERFACE_H

#include "database.h"

//用户界面类
class userinterface
{
private:
    database *emp_database;
public:
    userinterface();
    ~userinterface();

    //运行与交互主函数
    bool running();

    //功能函数，欢迎，退出，增删查改，排序，增和改考虑要按照不同类型雇员进行扩展
    void welcome();
    bool searchEmp();
    bool insertEmp();
    bool deleteEmp();
    bool updateEmp();
    bool sortEmp();
    void quit();
    //专门用于设置销售员与销售经理关系的界面
    bool setRelation();
    void pause();
};

#endif

/*****
FileName:database.cpp

```

Copyright: Tsinghua University

Author: Huaizhou Tao

Date:2014-10-02

Description: database类的定义

```
*****/
#include <iostream>
#include <fstream>
#include "database.h"
using namespace std;

//Summary: 构造函数
//
//Parameters:
//      None
//Return: None
//Detail:
database::database()
{
    mngList.makeEmpty();
    techList.makeEmpty();
    saleList.makeEmpty();
    salemngList.makeEmpty();
}

//Summary: 析构函数
//
//Parameters:
//      None
//Return: None
//Detail:
database::~~database()
{
    mngList.makeEmpty();
    techList.makeEmpty();
    saleList.makeEmpty();
    salemngList.makeEmpty();
}

//Summary: 从文件中读取
//
//Parameters:
//      None
//Return: None
```



```

//Detail:
void database::load()
{
    try
    {
        loadManager("info_manager.txt");
        loadSaleManager("info_salemanager.txt");
        loadTechnician("info_technician.txt");
        loadSalesman("info_salesman.txt");
    }
    catch(FileException e)
    {
        throw e;
    }
}

//Summary: 向文件中保存
//
//Parameters:
//      None
//Return: None
//Detail:
void database::save()
{
    try
    {
        saveManager("info_manager.txt");
        saveSaleManager("info_salemanager.txt");
        saveTechnician("info_technician.txt");
        saveSalesman("info_salesman.txt");
    }
    catch(FileException e)
    {
        throw e;
    }
}

//Summary: 从文件中读取经理信息
//
//Parameters:
//      fileName 文件名
//Return: None
//Detail:
void database::loadManager(const string &fileName)

```

```

{
    ifstream in(fileName.c_str(), ios::in);
    node<manager> *mng;
    //打开文件成功
    if(in)
    {
        while(!in.eof())
        {
            mng = new node<manager>;
            if(in>>mng->data)
            {
                mngList.insert_end(mng);
            }
            else
            {
                FileException e(fileName, "operate", "read");
                throw e;
            }
        }
    }
    else
    {
        FileException e(fileName, "open", "read");
        throw e;
    }

    in.close();
}

```

//Summary: 从文件中读取技术员信息

//

//Parameters:

// fileName 文件名

//Return: None

//Detail:

void database::loadTechnician(const string &fileName)

```

{
    ifstream in(fileName.c_str(), ios::in);
    node<technician> *tech;
    //打开文件成功
    if(in)
    {
        while(!in.eof())
        {

```

```

        tech = new node<technician>;
        if(in>>tech->data)
        {
            techList.insert_end(tech);
        }
        else
        {
            FileException e(fileName, "operate", "read");
            throw e;
        }
    }
}
else
{
    FileException e(fileName, "open", "read");
    throw e;
}

in.close();
}

```

//Summary: 从文件中读取销售员信息

//

//Parameters:

// fileName 文件名

//Return: None

//Detail:

```

void database::loadSalesman(const string &fileName)
{

```

```

    ifstream in(fileName.c_str(), ios::in);

```

```

    node<salesman> *sale;

```

```

    node<salemanager> *salemng;

```

```

    int saleMngNo;

```

//打开文件成功

```

    if(in)
    {

```

```

        while(!in.eof())
        {

```

```


```

```

            sale = new node<salesman>;

```

```

            if(in>>sale->data>>saleMngNo)
            {

```

```


```

//TODO: 关联销售经理

```

            salemng = salemngList.getFirst();

```

```

            if(salemng != NULL)

```

```

        {
            do
            {
                if(salemng->data.getEmpNo() == saleMngNo)
                {
                    sale->data.setBoss(&(salemng->data));
                    break;
                }
                salemng = salemngList.getNext();
            }while(salemng != NULL);
        }

        saleList.insert_end(sale);
    }
    else
    {
        FileNotFoundException e(fileName, "operate", "read");
        throw e;
    }
}
}
else
{
    FileNotFoundException e(fileName, "open", "read");
    throw e;
}

in.close();
}

```

//Summary: 从文件中读取销售经理信息

//

//Parameters:

// fileName 文件名

//Return: None

//Detail:

void database::loadSaleManager(const string &fileName)

```

{
    ifstream in(fileName.c_str(), ios::in);
    node<salemanager> *salemng;
    //打开文件成功
    if(in)
    {
        while(!in.eof())

```

```

    {
        salemng = new node<salemanager>;
        if(in>>salemng->data)
        {
            salemngList.insert_end(salemng);
        }
        else
        {
            FileException e(fileName, "operate", "read");
            throw e;
        }
    }
}
else
{
    FileException e(fileName, "open", "read");
    throw e;
}

in.close();
}

```

//Summary: 向文件中保存经理信息

//

//Parameters:

// fileName 文件名

//Return: None

//Detail:

```

void database::saveManager(const string &fileName)
{
    ofstream out(fileName.c_str(), ios::out|ios::trunc);
    node<manager> *mng;
    //打开文件成功
    if(out)
    {
        mng = mngList.getFirst();
        while(mng != NULL)
        {
            if(!(out<<mng->data))
            {
                FileException e(fileName, "operate", "write");
                throw e;
            }
            mng = mngList.getNext();
        }
    }
}

```

```

        }
    }
    else
    {
        FileException e(fileName, "open", "write");
        throw e;
    }

    out.close();
}

//Summary: 向文件中保存技术员信息
//
//Parameters:
//    fileName 文件名
//Return: None
//Detail:
void database::saveTechnician(const string &fileName)
{
    ofstream out(fileName.c_str(), ios::out|ios::trunc);
    node<technician> *tech;
    //打开文件成功
    if(out)
    {
        tech = techList.getFirst();
        while(tech != NULL)
        {
            if(!(out<<tech->data))
            {
                FileException e(fileName, "operate", "write");
                throw e;
            }
            tech = techList.getNext();
        }
    }
    else
    {
        FileException e(fileName, "open", "write");
        throw e;
    }

    out.close();
}

```

```

//Summary: 向文件中保存销售员信息
//
//Parameters:
//      fileName 文件名
//Return: None
//Detail:
void database::saveSalesman(const string &fileName)
{
    ofstream out(fileName.c_str(), ios::out|ios::trunc);
    node<salesman> *sale;
    //打开文件成功
    if(out)
    {
        sale = saleList.getFirst();
        while(sale != NULL)
        {
            if(!(out<<sale->data))
            {
                FileException e(fileName, "operate", "write");
                throw e;
            }
            sale = saleList.getNext();
        }
    }
    else
    {
        FileException e(fileName, "open", "write");
        throw e;
    }

    out.close();
}

```

```

//Summary: 向文件中保存销售经理信息
//
//Parameters:
//      fileName 文件名
//Return: None
//Detail:
void database::saveSaleManager(const string &fileName)
{
    ofstream out(fileName.c_str(), ios::out|ios::trunc);
    node<salemanager> *salemg;
    //打开文件成功

```

```

if(out)
{
    salemng = salemngList.getFirst();
    while(salemng != NULL)
    {
        if(!(out<<salemng->data))
        {
            FileNotFoundException e(fileName, "operate", "write");
            throw e;
        }
        salemng = salemngList.getNext();
    }
}
else
{
    FileNotFoundException e(fileName, "open", "write");
    throw e;
}

out.close();
}

```

//Summary: 以姓名为准删除雇员，可能有多个选择
//

//Parameters:

// name 雇员的姓名

//Return: bool 是否成功

//Detail:

```

bool database::deleteEmployee(const string &name)
{

```

```

    node<manager> *mng;
    node<technician> *tech;
    node<salesman> *sale;
    node<salemanager> *salemng;
    bool flag = false;

```

```

    mng = mngList.getFirst();
    if(mng != NULL)
    {
        do
        {
            if(mng->data.getName() == name)
            {
                mng->data.printInfo();
            }

```



```

        cout<<"按Y/y确认删除: ";
        char input;
        cin>>input;
        if((input == 'y')||(input == 'Y'))
        {
            mngList.remove(mng);
            flag = true;
            cout<<"删除成功。"<<endl;
        }
        else
        {
            cout<<"放弃删除。"<<endl;
        }
    }
    mng = mngList.getNext();
}while(mng != NULL);
}

tech = techList.getFirst();
if(tech != NULL)
{
    do
    {
        if(tech->data.getName() == name)
        {
            tech->data.printInfo();
            cout<<"按Y/y确认删除: ";
            char input;
            cin>>input;
            if((input == 'y')||(input == 'Y'))
            {
                techList.remove(tech);
                flag = true;
                cout<<"删除成功。"<<endl;
            }
            else
            {
                cout<<"放弃删除。"<<endl;
            }
        }
        tech = techList.getNext();
    }while(tech != NULL);
}

```

```

sale = saleList.getFirst();
if(sale != NULL)
{
    do
    {
        if(sale->data.getName() == name)
        {
            sale->data.printInfo();
            cout<<"按Y/y确认删除: ";
            char input;
            cin>>input;
            if((input == 'y')||(input == 'Y'))
            {
                saleList.remove(sale);
                flag = true;
                cout<<"删除成功。"<<endl;
            }
            else
            {
                cout<<"放弃删除。"<<endl;
            }
        }
        sale = saleList.getNext();
    }while(sale != NULL);
}

```

```

salemg = salemgList.getFirst();
if(salemg != NULL)
{
    do
    {
        if(salemg->data.getName() == name)
        {
            salemg->data.printInfo();
            cout<<"按Y/y确认删除: ";
            char input;
            cin>>input;
            if((input == 'y')||(input == 'Y'))
            {
                sale = saleList.getFirst();
                if(sale != NULL)
                {
                    do

```

```

        {
            if(sale->data.getBoss()->getEmpNo() ==
salemng->data.getEmpNo())
            {
                sale->data.setBoss(NULL);
            }
            sale = saleList.getNext();
        }while(sale != NULL);
    }

    salemngList.remove(salemng);
    flag = true;
    cout<<"删除成功。"<<endl;
}
else
{
    cout<<"放弃删除。"<<endl;
}
}
    salemng = salemngList.getNext();
}while(salemng != NULL);
}
return flag;
}

```

//Summary: 以编号为准删除雇员

//

//Parameters:

// empNo 雇员的编号

//Return: bool 是否成功

//Detail:

bool database::deleteEmployee(int empNo)

```

{
    node<manager> *mng;
    node<technician> *tech;
    node<salesman> *sale;
    node<salemanager> *salemng;
    bool flag = false;

    mng = mngList.getFirst();
    if(mng != NULL)
    {
        do
        {

```

```

        if(mng->data.getEmpNo() == empNo)
        {
            flag = true;
            mng->data.printInfo();
            cout<<"按Y/y确认删除: ";
            char input;
            cin>>input;
            if((input == 'y') || (input == 'Y'))
            {
                mngList.remove(mng);
                cout<<"删除成功。"<<endl;
            }
            else
            {
                cout<<"放弃删除。"<<endl;
            }
        }
        mng = mngList.getNext();
    }while(mng != NULL);
}

tech = techList.getFirst();
if(tech != NULL)
{
    do
    {
        if(tech->data.getEmpNo() == empNo)
        {
            flag = true;
            tech->data.printInfo();
            cout<<"按Y/y确认删除: ";
            char input;
            cin>>input;
            if((input == 'y') || (input == 'Y'))
            {
                techList.remove(tech);
                cout<<"删除成功。"<<endl;
            }
            else
            {
                cout<<"放弃删除。"<<endl;
            }
        }
        tech = techList.getNext();
    }
}

```

```

    }while(tech != NULL);
}

sale = saleList.getFirst();
if(sale != NULL)
{
    do
    {
        if(sale->data.getEmpNo() == empNo)
        {
            flag = true;
            sale->data.printInfo();
            cout<<"按Y/y确认删除: ";
            char input;
            cin>>input;
            if((input == 'y')||(input == 'Y'))
            {
                saleList.remove(sale);
                cout<<"删除成功。"<<endl;
            }
            else
            {
                cout<<"放弃删除。"<<endl;
            }
        }
        sale = saleList.getNext();
    }while(sale != NULL);
}

```

```

saleMng = saleMngList.getFirst();
if(saleMng != NULL)
{
    do
    {
        if(saleMng->data.getEmpNo() == empNo)
        {
            flag = true;
            saleMng->data.printInfo();
            cout<<"按Y/y确认删除: ";
            char input;
            cin>>input;
            if((input == 'y')||(input == 'Y'))
            {

```

```

        sale = saleList.getFirst();
        if(sale != NULL)
        {
            do
            {
                if(sale->data.getBoss()->getEmpNo() ==
saleMng->data.getEmpNo())
                {
                    sale->data.setBoss(NULL);
                }
                sale = saleList.getNext();
            }while(sale != NULL);
        }

        saleMngList.remove(saleMng);
        cout<<"删除成功。"<<endl;
    }
    else
    {
        cout<<"放弃删除。"<<endl;
    }
}

saleMng = saleMngList.getNext();
}while(saleMng != NULL);
}
return flag;
}

```

//Summary: 新增经理

//

//Parameters:

// None

//Return: None

//Detail:

void database::addManager()

```

{
    node<manager> *mng = new node<manager>;
    mng->data.updateInfo();
    mngList.insert_end(mng);
}

```

//Summary: 新增技术员

//

//Parameters:

```

//      None
//Return: None
//Detail:
void database::addTechnician()
{
    node<technician> *tech = new node<technician>;
    tech->data.updateInfo();
    techList.insert_end(tech);
}

//Summary: 新增销售员
//
//Parameters:
//      None
//Return: None
//Detail:
void database::addSalesman()
{
    node<salesman> *sale = new node<salesman>;
    sale->data.updateInfo();
    saleList.insert_end(sale);
}

//Summary: 新增销售经理
//
//Parameters:
//      None
//Return: None
//Detail:
void database::addSaleManager()
{
    node<salemanager> *salemng = new node<salemanager>;
    salemng->data.updateInfo();
    salemngList.insert_end(salemng);
}

//Summary: 新增雇员
//
//Parameters:
//      EmployeeType 雇员的具体类型
//Return: None
//Detail: 这是对外接口，上面的函数是对内实现
void database::addEmployee(EmployeeType e)
{

```

```

switch(e)
{
case Manager:
    addManager();
    break;
case Technician:
    addTechnician();
    break;
case Salesman:
    addSalesman();
    break;
case SaleManager:
    addSaleManager();
    break;
default:
    cout<<"输入类型错误."<<endl;
    break;
}
}

//Summary: 输出基本信息
//
//Parameters:
//    empNo 雇员的编号
//Return: None
//Detail:
void database::detailInfo(int empNo)
{
    node<manager> *mng;
    node<technician> *tech;
    node<salesman> *sale;
    node<salemanager> *salemng;

    mng = mngList.getFirst();
    if(mng != NULL)
    {
        do
        {
            if(mng->data.getEmpNo() == empNo)
            {
                mng->data.printInfo();
            }
            mng = mngList.getNext();
        }while(mng != NULL);
    }
}

```



```

    }

    tech = techList.getFirst();
    if(tech != NULL)
    {
        do
        {
            if(tech->data.getEmpNo() == empNo)
            {
                tech->data.printInfo();
            }
            tech = techList.getNext();
        }while(tech != NULL);
    }

    sale = saleList.getFirst();
    if(sale != NULL)
    {
        do
        {
            if(sale->data.getEmpNo() == empNo)
            {
                sale->data.printInfo();
            }
            sale = saleList.getNext();
        }while(sale != NULL);
    }

    salemng = salemngList.getFirst();
    if(salemng != NULL)
    {
        do
        {
            if(salemng->data.getEmpNo() == empNo)
            {
                salemng->data.printInfo();
            }
            salemng = salemngList.getNext();
        }while(salemng != NULL);
    }
}

//Summary: 显示某一类的所有雇员
//

```

```

//Parameters:
//      EmployeeType 雇员类型
//Return: int 雇员数目
//Detail: 选择Employee类，显示全部雇员
int database::showAll(EmployeeType e)
{
    node<manager> *mng;
    node<technician> *tech;
    node<salesman> *sale;
    node<salemanager> *salemng;
    int count = 0;

    switch(e)
    {
    case Employee:
        mng = mngList.getFirst();
        if(mng != NULL)
        {
            do
            {
                mng->data.printInfo();
                count++;
                mng = mngList.getNext();
            }while(mng != NULL);
        }
        tech = techList.getFirst();
        if(tech != NULL)
        {
            do
            {
                tech->data.printInfo();
                count++;
                tech = techList.getNext();
            }while(tech != NULL);
        }
        sale = saleList.getFirst();
        if(sale != NULL)
        {
            do
            {
                sale->data.printInfo();
                count++;
                sale = saleList.getNext();
            }while(sale != NULL);
        }
    }
}

```

```

    }
    salemng = salemngList.getFirst();
    if(salemng != NULL)
    {
        do
        {
            salemng->data.printInfo();
            count++;
            salemng = salemngList.getNext();
        }while(salemng != NULL);
    }
    break;
case Manager:
    mng = mngList.getFirst();
    if(mng != NULL)
    {
        do
        {
            mng->data.printInfo();
            count++;
            mng = mngList.getNext();
        }while(mng != NULL);
    }
    break;
case Technician:
    tech = techList.getFirst();
    if(tech != NULL)
    {
        do
        {
            tech->data.printInfo();
            count++;
            tech = techList.getNext();
        }while(tech != NULL);
    }
    break;
case Salesman:
    sale = saleList.getFirst();
    if(sale != NULL)
    {
        do
        {
            sale->data.printInfo();
            count++;

```

```

        sale = saleList.getNext();
    }while(sale != NULL);
}
break;
case SaleManager:
    salemng = salemngList.getFirst();
    if(salemng != NULL)
    {
        do
        {
            salemng->data.printInfo();
            count++;
            salemng = salemngList.getNext();
        }while(salemng != NULL);
    }
    break;
default:
    break;
}
return count;
}

```

//Summary: 按编号更新雇员信息

//

//Parameters:

// empNo 雇员编号

//Return: bool 是否找到目标并更新成功

//Detail:

```

bool database::updateEmployee(int empNo)
{
    node<manager> *mng;
    node<technician> *tech;
    node<salesman> *sale;
    node<salemanager> *salemng;
    bool flag = false;

    mng = mngList.getFirst();
    if(mng != NULL)
    {
        do
        {
            if(mng->data.getEmpNo() == empNo)
            {
                flag = true;
            }
        }
    }
}

```

```

        mng->data.printInfo();
        cout<<"按Y/y确认更新：";
        char input;
        cin>>input;
        if((input == 'y')||(input == 'Y'))
        {
            mng->data.updateInfo();
            cout<<"更新成功。"<<endl;
        }
        else
        {
            cout<<"放弃更新。"<<endl;
        }
    }
    mng = mngList.getNext();
}while(mng != NULL);
}

tech = techList.getFirst();
if(tech != NULL)
{
    do
    {
        if(tech->data.getEmpNo() == empNo)
        {
            flag = true;
            tech->data.printInfo();
            cout<<"按Y/y确认更新：";
            char input;
            cin>>input;
            if((input == 'y')||(input == 'Y'))
            {
                tech->data.updateInfo();
                cout<<"更新成功。"<<endl;
            }
            else
            {
                cout<<"放弃更新。"<<endl;
            }
        }
        tech = techList.getNext();
    }while(tech != NULL);
}

```

```

sale = saleList.getFirst();
if(sale != NULL)
{
    do
    {
        if(sale->data.getEmpNo() == empNo)
        {
            flag = true;
            sale->data.printInfo();
            cout<<"按Y/y确认更新: ";
            char input;
            cin>>input;
            if((input == 'y')||(input == 'Y'))
            {
                sale->data.updateInfo();
                cout<<"更新成功。"<<endl;
            }
            else
            {
                cout<<"放弃更新。"<<endl;
            }
        }
        sale = saleList.getNext();
    }while(sale != NULL);
}

```

```

salemg = salemngList.getFirst();
if(salemg != NULL)
{
    do
    {
        if(salemg->data.getEmpNo() == empNo)
        {
            flag = true;
            salemng->data.printInfo();
            cout<<"按Y/y确认更新: ";
            char input;
            cin>>input;
            if((input == 'y')||(input == 'Y'))
            {
                salemng->data.updateInfo();
                cout<<"更新成功。"<<endl;
            }
            else

```

```

        {
            cout<<"放弃更新。"<<endl;
        }
    }
    salemng = salemngList.getNext();
}while(salemng != NULL);
}
return flag;
}

//Summary: 按编号查找雇员
//
//Parameters:
//      empNo 雇员编号
//Return: int 返回查到的雇员数目
//Detail:
int database::search(int empNo)
{
    node<manager> *mng;
    node<technician> *tech;
    node<salesman> *sale;
    node<salemanager> *salemng;
    int count = 0;

    mng = mngList.getFirst();
    if(mng != NULL)
    {
        do
        {
            if(mng->data.getEmpNo() == empNo)
            {
                mng->data.printInfo();
                count++;
            }
            mng = mngList.getNext();
        }while(mng != NULL);
    }

    tech = techList.getFirst();
    if(tech != NULL)
    {
        do
        {
            if(tech->data.getEmpNo() == empNo)

```

```

        {
            tech->data.printInfo();
            count++;
        }
        tech = techList.getNext();
    }while(tech != NULL);
}

sale = saleList.getFirst();
if(sale != NULL)
{
    do
    {
        if(sale->data.getEmpNo() == empNo)
        {
            sale->data.printInfo();
            count++;
        }
        sale = saleList.getNext();
    }while(sale != NULL);
}

salemng = salemngList.getFirst();
if(salemng != NULL)
{
    do
    {
        if(salemng->data.getEmpNo() == empNo)
        {
            salemng->data.printInfo();
            count++;
        }
        salemng = salemngList.getNext();
    }while(salemng != NULL);
}
return count;
}

```

//Summary: 按姓名查找雇员

//

//Parameters:

// name 雇员姓名

//Return: int 查到的雇员数目

//Detail:


```

int database::search(const string &name)
{
    node<manager> *mng;
    node<technician> *tech;
    node<salesman> *sale;
    node<salemanager> *salemng;
    int count = 0;

    mng = mngList.getFirst();
    if(mng != NULL)
    {
        do
        {
            if(mng->data.getName() == name)
            {
                mng->data.printInfo();
                count++;
            }
            mng = mngList.getNext();
        }while(mng != NULL);
    }

    tech = techList.getFirst();
    if(tech != NULL)
    {
        do
        {
            if(tech->data.getName() == name)
            {
                tech->data.printInfo();
                count++;
            }
            tech = techList.getNext();
        }while(tech != NULL);
    }

    sale = saleList.getFirst();
    if(sale != NULL)
    {
        do
        {
            if(sale->data.getName() == name)
            {
                sale->data.printInfo();
            }
        }while(sale != NULL);
    }
}

```

```

        count++;
    }
    sale = saleList.getNext();
}while(sale != NULL);
}

saalemng = saalemngList.getFirst();
if(saalemng != NULL)
{
    do
    {
        if(saalemng->data.getName() == name)
        {
            saalemng->data.printInfo();
            count++;
        }
        saalemng = saalemngList.getNext();
    }while(saalemng != NULL);
}
return count;
}

//Summary: 工资排序函数
//
//Parameters:
//      EmployeeType 雇员类型
//      direction    升序/降序
//Return: None
//Detail:
void database::sortByPay(EmployeeType e, int direction)
{
    employee **head;
    int length = 0;
    switch(e)
    {
        case Employee:
            length = mngList.size() + techList.size() + saleList.size() +
saalemngList.size();
            if(length > 0)
            {
                head = new employee*[length];
                int cnt = 0;

                node<manager>* m = mngList.getFirst();

```

```

if(m!=NULL)
{
    do
    {
        head[cnt] = &m->data;
        m = mngList.getNext();
        cnt++;
    }while(m != NULL);
}
node<technician>* t = techList.getFirst();
if(t!=NULL)
{
    do
    {
        head[cnt] = &t->data;
        t = techList.getNext();
        cnt++;
    }while(t != NULL);
}
node<salesman>* s = saleList.getFirst();
if(s!=NULL)
{
    do
    {
        head[cnt] = &s->data;
        s = saleList.getNext();
        cnt++;
    }while(s != NULL);
}
node<salemanager>* sm = salemngList.getFirst();
if(sm!=NULL)
{
    do
    {
        head[cnt] = &sm->data;
        sm = salemngList.getNext();
        cnt++;
    }while(sm != NULL);
}
sortCustom(head, length, direction, 1);
for(cnt = 0; cnt < length; cnt++)
    head[cnt]->printInfo();
delete []head;
}

```

```

else
{
    cout<<"员工数量为。"<<endl;
}
break;
case Manager:
length = mngList.size();
if(length > 0)
{
    head = new employee*[length];
    node<manager>* tmp = mngList.getFirst();
    int cnt = 0;
    do
    {
        head[cnt] = &tmp->data;
        tmp = mngList.getNext();
        cnt++;
    }while(tmp != NULL);
    sortCustom(head, length, direction, 1);
    for(cnt = 0; cnt < length; cnt++)
        head[cnt]->printInfo();
    delete []head;
}
else
{
    cout<<"经理员工数量为。"<<endl;
}
break;
case Technician:
length = techList.size();
if(length > 0)
{
    head = new employee*[length];
    node<technician>* tmp = techList.getFirst();
    int cnt = 0;
    do
    {
        head[cnt] = &tmp->data;
        tmp = techList.getNext();
        cnt++;
    }while(tmp != NULL);
    sortCustom(head, length, direction, 1);
    for(cnt = 0; cnt < length; cnt++)
        head[cnt]->printInfo();
}

```

```

        delete []head;
    }
    else
    {
        cout<<"技术员工数量为。"<<endl;
    }
    break;
case Salesman:
    length = saleList.size();
    if(length > 0)
    {
        head = new employee*[length];
        node<salesman>* tmp = saleList.getFirst();
        int cnt = 0;
        do
        {
            head[cnt] = &tmp->data;
            tmp = saleList.getNext();
            cnt++;
        }while(tmp != NULL);
        sortCustom(head, length, direction, 1);
        for(cnt = 0; cnt < length; cnt++)
            head[cnt]->printInfo();
        delete []head;
    }
    else
    {
        cout<<"销售员工数量为。"<<endl;
    }
    break;
case SaleManager:
    length = salemngList.size();
    if(length > 0)
    {
        head = new employee*[length];
        node<salemanager>* tmp = salemngList.getFirst();
        int cnt = 0;
        do
        {
            head[cnt] = &tmp->data;
            tmp = salemngList.getNext();
            cnt++;
        }while(tmp != NULL);
        sortCustom(head, length, direction, 1);
    }
}

```

```

        for(cnt = 0; cnt < length; cnt++)
            head[cnt]->printInfo();
        delete []head;
    }
    else
    {
        cout<<"销售经理员工数量为。"<<endl;
    }
    break;
default:
    cout<<"输入类型有误。"<<endl;
    break;
}
}

//Summary: 工号排序函数
//
//Parameters:
//      EmployeeType 雇员类型
//      direction    升序/降序
//Return: None
//Detail:
void database::sortByNo(EmployeeType e, int direction)
{
    employee **head;
    int length = 0;
    switch(e)
    {
    case Employee:
        length = mngList.size() + techList.size() + saleList.size() +
saleMngList.size();
        if(length > 0)
        {
            head = new employee*[length];
            int cnt = 0;

            node<manager>* m = mngList.getFirst();
            if(m!=NULL)
            {
                do
                {
                    head[cnt] = &m->data;
                    m = mngList.getNext();
                    cnt++;
                }
            }
        }
    }
}

```

```

        }while(m != NULL);
    }
    node<technician>* t = techList.getFirst();
    if(t!=NULL)
    {
        do
        {
            head[cnt] = &t->data;
            t = techList.getNext();
            cnt++;
        }while(t != NULL);
    }
    node<salesman>* s = saleList.getFirst();
    if(s!=NULL)
    {
        do
        {
            head[cnt] = &s->data;
            s = saleList.getNext();
            cnt++;
        }while(s != NULL);
    }
    node<salemanager>* sm = salemngList.getFirst();
    if(sm!=NULL)
    {
        do
        {
            head[cnt] = &sm->data;
            sm = salemngList.getNext();
            cnt++;
        }while(sm != NULL);
    }
    sortCustom(head, length, direction, 0);
    for(cnt = 0; cnt < length; cnt++)
        head[cnt]->printInfo();
    delete []head;
}
else
{
    cout<<"员工数量为。"<<endl;
}
break;
case Manager:
    length = mngList.size();

```

```

if(length > 0)
{
    head = new employee*[length];
    node<manager>* tmp = mngList.getFirst();
    int cnt = 0;
    do
    {
        head[cnt] = &tmp->data;
        tmp = mngList.getNext();
        cnt++;
    }while(tmp != NULL);
    sortCustom(head, length, direction, 0);
    for(cnt = 0; cnt < length; cnt++)
        head[cnt]->printInfo();
    delete []head;
}
else
{
    cout<<"经理员工数量为。"<<endl;
}
break;
case Technician:
    length = techList.size();
    if(length > 0)
    {
        head = new employee*[length];
        node<technician>* tmp = techList.getFirst();
        int cnt = 0;
        do
        {
            head[cnt] = &tmp->data;
            tmp = techList.getNext();
            cnt++;
        }while(tmp != NULL);
        sortCustom(head, length, direction, 0);
        for(cnt = 0; cnt < length; cnt++)
            head[cnt]->printInfo();
        delete []head;
    }
    else
    {
        cout<<"技术员工数量为。"<<endl;
    }
    break;

```



```

case Salesman:
    length = saleList.size();
    if(length > 0)
    {
        head = new employee*[length];
        node<salesman>* tmp = saleList.getFirst();
        int cnt = 0;
        do
        {
            head[cnt] = &tmp->data;
            tmp = saleList.getNext();
            cnt++;
        }while(tmp != NULL);
        sortCustom(head, length, direction, 0);
        for(cnt = 0; cnt < length; cnt++)
            head[cnt]->printInfo();
        delete []head;
    }
    else
    {
        cout<<"销售员工数量为。"<<endl;
    }
    break;
case SaleManager:
    length = salemngList.size();
    if(length > 0)
    {
        head = new employee*[length];
        node<salemanager>* tmp = salemngList.getFirst();
        int cnt = 0;
        do
        {
            head[cnt] = &tmp->data;
            tmp = salemngList.getNext();
            cnt++;
        }while(tmp != NULL);
        sortCustom(head, length, direction, 0);
        for(cnt = 0; cnt < length; cnt++)
            head[cnt]->printInfo();
        delete []head;
    }
    else
    {
        cout<<"销售经理员工数量为。"<<endl;
    }

```

```

        }
        break;
    default:
        cout<<"输入类型有误。"<<endl;
        break;
    }
}

//Summary: 排序函数
//
//Parameters:
//    head        数组地址
//    length       数组长度
//    keycol       主列: 工号/工资
//    direction    升序/降序
//Return: None
//Detail:
void database::sortCustom(employee **head, int length, int direction, int keycol)
{
    employee *tmp;
    bool flag;
    do
    {
        flag = false;
        for(int i = 0; i < length - 1; i++)
        {
            if(compare(head[i], head[i+1], direction, keycol) < 0)
            {
                tmp = head[i];
                head[i] = head[i+1];
                head[i+1] = tmp;
                flag = true;
            }
        }
    }while(flag);
}

//Summary: 比较函数
//
//Parameters:
//    e1           雇员
//    e2           雇员
//    keycol       主列: 工号/工资
//    direction    升序/降序

```

```

//Return: None
//Detail:
int database::compare(employee *e1, employee *e2, int direction, int keycol)
{
    int compareNo = e1->getEmpNo() - e2->getEmpNo();
    int comparePay = e1->getAccumPay() - e2->getAccumPay();
    if(keycol == 0)
    {
        return compareNo * direction;
    }
    else
    {
        return comparePay * direction;
    }
}

```

//Summary: 设置销售经理与销售员的关系

//

//Parameters:

// salesmanNo 销售员编号

// saleManagerNo 销售经理编号

//Return: bool 设置是否成功

//Detail:

```

bool database::setRelation(int salesmanNo, int saleManagerNo)
{
    salesman *s = NULL;
    salemanager *sm = NULL;

    node<salesman>* tmpS = saleList.getFirst();
    if(tmpS!=NULL)
    {
        do
        {
            if(tmpS->data.getEmpNo() == salesmanNo)
            {
                s = &tmpS->data;
                break;
            }
            tmpS = saleList.getNext();
        }while(tmpS != NULL);
    }

    node<salemanager>* tmpSM = salemgList.getFirst();
    if(tmpSM!=NULL)

```

```

    {
        do
        {
            if(tmpSM->data.getEmpNo() == saleManagerNo)
            {
                sm = &tmpSM->data;
                break;
            }
            tmpSM = salemngList.getNext();
        }while(tmpSM != NULL);
    }

    if(s != NULL && sm != NULL)
    {
        s->setBoss(sm);
        return true;
    }

    return false;
}

/*****
FileName:employee.cpp
Copyright: Tsinghua University
Author: Huaizhou Tao
Date:2014-10-02
Description: employee类的定义

*****/
#include <iostream>
#include <string>
#include "employee.h"
using namespace std;

//设置静态变量的初始值
int employee::currentEmpNo = 2014001;

//Summary: 输出雇员的基本信息
//
//Parameters:
//      None
//Return: None
//Detail: 该函数首先输出一条提示信息，然后按照
//      编号、等级与月薪的顺序输出雇员的基本信息

```

```

void employee::printInfo()
{
    cout<<"编号\t姓名\t等级\t月薪"<<endl;
    cout<<individualEmpNo<<' \t' <<name<<' \t' <<grade<<' \t' <<accumPay<<endl;
}

//Summary: 调级函数
//
//Parameters:
//      gradeUp 级别的变化值
//Return: bool   是否成功
//Detail: 可以降级，但最低一级
bool employee::promote(int gradeUp)
{
    if(grade + gradeUp > 0)
    {
        grade += gradeUp;
        return true;
    }
    return false;
}

//Summary: 无参构造函数
//
//Parameters:
//      None
//Return: None
//Detail: 无参构造函数首先完成新雇员的自动编号功能
//      再将雇员的等级设为，月薪设为
employee::employee()
{
    individualEmpNo = currentEmpNo;
    currentEmpNo++;
    grade = 1;
    accumPay = 3000;
    name.clear();
}

//Summary: 无参构造函数
//
//Parameters:
//      inputEmpNo: 输入的雇员编号
//      inputGrade: 输入的雇员等级
//      inputPay:   输入的雇员月薪

```

```

//      inputName:  输入的雇员姓名
//Return: None
//Detail: 有参构造函数首先完成新雇员的自动编号功能
//      之后根据输入的参数设置雇员等级与月薪
//注:      未添加合法性判断, 请勿输入非正数
employee::employee(int inputEmpNo, int inputGrade, int inputPay, string inputName)
{
    setEmpNo(inputEmpNo);
    grade = inputGrade;
    accumPay = inputPay;
    name = inputName;
}

//Summary: 析构函数
//
//Parameters:
//      None
//Return: None
//Detail: 释放对象, 并输出提示信息
employee::~employee()
{
    //cout<<"欢迎使用, 再见!"<<endl;
}

//Summary: 更新信息函数
//
//Parameters:
//      None
//Return: None
//Detail: 更新该雇员的个人信息 (除编号外)
void employee::updateInfo()
{
    cout<<"输入编号为"<<individualEmpNo<<" 的雇员的姓名、等级、月薪: "<<endl;
    cin>>name;
    cin>>grade;
    cin>>accumPay;
}

//Summary: 流输入操作符重载函数
//
//Parameters:
//      input: 输入流
//      e:      目标雇员对象
//Return: istream& 返回输入流本身

```

```

//Detail: 重载函数首先给出输入提示, 之后从流中输入目标对象的等级与月薪
//注:      未添加合法性判断, 请勿输入非正数
istream& operator >> (istream& input, employee& e)
{
    //cout<<"输入编号为"<<e.individualEmpNo<<" 的雇员的姓名、等级、月薪: "<<endl;
    int inputEmpNo;
    input>>inputEmpNo;
    e.setEmpNo(inputEmpNo);
    input>>e.name;
    input>>e.grade;
    input>>e.accumPay;
    return input;
}

//Summary: 流输出操作符重载函数
//
//Parameters:
//      output: 输出流
//      e:      目标雇员对象
//Return: ostream& 返回输出流本身
ostream& operator << (ostream &output, employee &e)
{
    cout<<e.individualEmpNo<<' \t' <<e.name<<' \t' <<e.grade<<' \t' <<e.accumPay<<endl;
    return output;
}

//Summary: 设置编号函数
//
//Parameters:
//      inputEmpNo 雇员编号
//Return: None
void employee::setEmpNo(int inputEmpNo)
{
    if(inputEmpNo <= 0)
    {
        individualEmpNo = currentEmpNo;
        currentEmpNo++;
    }
    else
    {
        if(inputEmpNo >= currentEmpNo)
            currentEmpNo = inputEmpNo + 1;
        if(individualEmpNo == currentEmpNo - 1 && inputEmpNo < individualEmpNo)
            currentEmpNo--;
    }
}

```

```

        individualEmpNo = inputEmpNo;
    }
}

/*****
FileName:linkedList.cpp
Copyright: Tsinghua University
Author: Huaizhou Tao
Date:2014-10-02
Description: linkedlist类的定义

*****/

#include "linkedList.h"

//Summary: 无参构造函数
//
//Parameters:
//      None
//Return: None
template<class T>
linkedList<T>::linkedList()
{
    head = current = NULL;
}

//Summary: 拷贝构造函数
//
//Parameters:
//      aplist
//Return: None
template<class T>
linkedList<T>::linkedList(const linkedList<T> &aplist)
{
    deepCopy(aplist);
}

//Summary: 析构函数
//
//Parameters:
//      None
//Return: None
template<class T>
linkedList<T>::~~linkedList()

```



```

{
    makeEmpty();
}

//Summary: 插入函数
//
//Parameters:
//      newNode 待插入元素
//Return: None
//Detail: 该函数在头部之前插入，插入后没有当前位置
template<class T>
void linkedlist<T>::insert(node<T> *newNode)
{
    current = NULL;
    newNode->next = head;
    head = newNode;
}

//Summary: 插入函数
//
//Parameters:
//      newNode 待插入元素
//Return: None
//Detail: 该函数在尾部插入新元素
template<class T>
void linkedlist<T>::insert_end(node<T> *newNode)
{
    current = NULL;
    node<T> *tail = head;
    newNode->next = NULL;
    if(tail == NULL)
    {
        head = newNode;
    }
    else
    {
        while(tail->next != NULL)
            tail = tail->next;
        tail->next = newNode;
    }
}

//Summary: 获得链表头的函数
//

```

```

//Parameters:
//      None
//Return: node<T>* 返回头结点
//Detail: 在链表为空时返回false
template<class T>
node<T>* linkedlist<T>::getFirst()
{
    if(head == NULL)
        return NULL;
    current = head;
    return head;
}

//Summary: 获得下一个数据
//
//Parameters:
//      None
//Return: node<T> * 返回结点
//Detail: current在执行后将指向下一个元素
template<class T>
node<T>* linkedlist<T>::getNext()
{
    if(current == NULL)
        return NULL;
    if(current->next == NULL)
    {
        current = NULL;
        return NULL;
    }
    current = current->next;
    return current;
}

//Summary: 查找一个数据
//
//Parameters:
//      element 目标数据
//Return: bool
//Detail:
template<class T>
bool linkedlist<T>::find(const T &element)
{
    node<T> *n;
    n = getFirst();

```

```

        if(n == NULL)
            return false;
        do
        {
            if(n->data == element)
                return true;
            n = getNext();
        }
        while(n != NULL);
        return false;
    }

```

//Summary: 检索一个数据

//

//Parameters:

// element 目标数据

//Return: bool

//Detail:

```
template<class T>
```

```
bool linkedlist<T>::retrieve(T &element)
```

```

{
    if(!find(element))
        return false;
    element = current->data;
    return true;
}

```

//Summary: 替换一个数据

//

//Parameters:

// newElement 目标数据

//Return: bool

//Detail: 将当前位置的数据替换

```
template<class T>
```

```
bool linkedlist<T>::replace(const T &newElement)
```

```

{
    if(current == NULL)
        return false;
    current->data = newElement;
    return true;
}

```

//Summary: 移除一个数据

//

```

//Parameters:
//      node 目标结点
//Return: bool
//Detail:
template<class T>
bool linkedlist<T>::remove(node<T> *n)
{
    current = NULL;
    if(head == NULL)
        return false;
    node<T> *tmp = head;
    if(head == n)
    {
        head = tmp->next;
        delete tmp;
        return true;
    }
    while(tmp->next != NULL)
    {
        if(tmp->next == n)
        {
            node<T> *ptr = tmp->next;
            tmp->next = ptr->next;
            delete ptr;
            return true;
        }
        tmp = tmp->next;
    }
    return false;
}

```

//Summary: 判断是否为空

//

//Parameters:

// None

//Return: bool

//Detail:

```

template<class T>
bool linkedlist<T>::isEmpty() const
{
    return head == NULL;
}

```

//Summary: 将链表清空

```

//
//Parameters:
//      None
//Return: None
//Detail:
template<class T>
void linkedlist<T>::makeEmpty()
{
    while(head != NULL)
    {
        current = head;
        head = head->next;
        delete current;
    }
    current = NULL;
}

//Summary: 获得链表大小
//
//Parameters:
//      None
//Return: int 链表元素数目
//Detail:
template<class T>
int linkedlist<T>::size()
{
    int size = 0;
    node<T> *tmp = head;
    while(tmp != NULL)
    {
        size++;
        tmp = tmp->next;
    }
    return size;
}

//Summary: "="运算符重载
//
//Parameters:
//      rlist 右操作符
//Return: this
//Detail:
template<class T>
linkedlist<T>& linkedlist<T>::operator =(const linkedlist<T> &rlist)

```

```

{
    if(this == &rlist)
        return *this;
    makeEmpty();
    deepCopy(rlist);
    return *this;
}

//Summary: 深拷贝函数
//
//Parameters:
//    original 原链表
//Return: None
//Detail:
template<class T>
void linkedlist<T>::deepCopy(const linkedlist<T> &original)
{
    head = current = NULL;
    if(original.head == NULL)
        return ;

    node<T> *copy = head = new node<T>;
    node<T> *origin = original.head;
    copy->data = origin->data;
    if(origin == original.current)
        current = copy;

    while(origin->next != NULL)
    {
        copy->next = new node<T>;
        origin = origin->next;
        copy = copy->next;
        copy->data = origin->data;
        if(origin == original.current)
            current = copy;
    }
    copy->next = NULL;
}

/*****
FileName:main.cpp
Copyright: Tsinghua University
Author: Huaizhou Tao
Date:2014-10-02

```

Description: 程序主函数文件

```

*****/
#include <iostream>
#include <fstream>
#include "employee.h"
#include "manager.h"
#include "technician.h"
#include "salesman.h"
#include "salemanager.h"
#include "database.h"
#include "userinterface.h"
using namespace std;

#define NUM_SALEMNG 2
#define NUM_TECHNICIAN 10
#define NUM_SALESMAN 10

//TODO:
//增加界面，增加功能函数：录入（文件、键盘）、存储（到文件）、查询（包括排序）、删除、
修改、退出
//存储数据结构，用对象链表

//Summary: 主函数
//
//Parameters:
//      None
//Return: int 一般为
int main()
{
    userinterface ui;

    while(ui.running());

    return 0;
}

/*****
FileName:manager.cpp
Copyright: Tsinghua University
Author: Huaizhou Tao
Date:2014-10-02
Description: manager类的定义

```

```

*****/
#include <iostream>
#include "manager.h"
using namespace std;

//Summary: 输出经理的基本信息
//
//Parameters:
//      None
//Return: None
//Detail: 与employee类的函数相比, 增加了岗位信息
void manager::printInfo()
{
    cout<<"岗位\t编号\t姓名\t等级\t月薪"<<endl;
    cout<<"经理
\t"<<getEmpNo()<<' \t'<<getName()<<' \t'<<getGrade()<<' \t'<<getAccumPay()<<endl;
}

//Summary: 计算工资
//
//Parameters:
//      None
//Return: int 工资的值
//Detail: 经理的工资是固定工资
int manager::PAY()
{
    setAccumPay(fixed_pay);
    return fixed_pay;
}

//Summary: 调级函数
//
//Parameters:
//      gradeUp 级别的变化值
//Return: bool 是否成功
//Detail: 1级固定工资元, 每提升一级, 工资增加元
bool manager::promote(int gradeUp)
{
    int grade_tmp = getGrade();
    if(grade_tmp + gradeUp > 0)
    {
        grade_tmp += gradeUp;
        fixed_pay = FIXED_PAY_MANAGER + PROMOTE_PAY_MANAGER * (grade_tmp - 1);
    }
}

```



```

        setGrade(grade_tmp);
        PAY();
        return true;
    }
    return false;
}

//Summary: 无参构造函数
//
//Parameters:
//    None
//Return: None
//Detail: 调用上级构造函数，默认设置等级为
manager::manager():employee(0, 1, FIXED_PAY_MANAGER, "")
{

}

//Summary: 有参构造函数
//
//Parameters:
//    inputEmpNo: 输入的雇员编号
//    inputGrade: 输入的雇员等级
//    inputName: 输入的雇员姓名
//Return: None
//Detail: 调用上级构造函数，设置等级为输入等级，月薪为固定月薪
manager::manager(int inputEmpNo, int inputGrade, string
inputName):employee(inputEmpNo, 1, FIXED_PAY_MANAGER, inputName)
{
    //TODO: 由于等级变化，进行调级与工资计算
    promote(inputGrade - 1);
}

//Summary: 析构函数
//
//Parameters:
//    None
//Return: None
manager::~~manager()
{

}

//Summary: 更新信息函数

```

```

//
//Parameters:
//      None
//Return: None
//Detail: 更新该雇员的个人信息（除编号外）
void manager::updateInfo()
{
    cout<<"输入编号为"<<getEmpNo()<<" 的雇员的姓名、等级: "<<endl;
    int tempGrade;
    string tempName;
    cin>>tempName;
    cin>>tempGrade;

    setName(tempName);
    //TODO: 由于等级变化, 进行调级与工资计算
    promote(tempGrade - getGrade());
}

//Summary: 流输入操作符重载函数
//
//Parameters:
//      input: 输入流
//      m:      目标经理对象
//Return: istream& 返回输入流本身
//Detail: 重载函数首先给出输入提示, 之后从流中输入目标对象的等级
//注:      未添加合法性判断, 请勿输入非正数; 月薪为固定, 不能输入
istream& operator >> (istream& input, manager& m)
{
    //cout<<"输入编号为"<<m.getEmpNo()<<" 的经理的姓名、等级: "<<endl;
    int tempEmpNo;
    int tempGrade;
    int tempPay;
    string tempName;
    input>>tempEmpNo;
    input>>tempName;
    input>>tempGrade;
    input>>tempPay;

    m.setEmpNo(tempEmpNo);
    m.setName(tempName);
    //TODO: 由于等级变化, 进行调级与工资计算
    m.promote(tempGrade - m.getGrade());

    return input;
}

```

```

}

//Summary: 流输出操作符重载函数
//
//Parameters:
//    output: 输出流
//    m:      目标经理对象
//Return: ostream& 返回输出流本身
ostream& operator << (ostream &output, manager &m)
{
    output<<endl<<m.getEmpNo()<<' \t'<<m.getName()<<' \t'<<m.getGrade()<<' \t'<<m.get
AccumPay();
    return output;
}

/*****
FileName:salemanager.cpp
Copyright: Tsinghua University
Author: Huaizhou Tao
Date:2014-10-02
Description: salemanager类的定义

*****/
#include <iostream>
#include "salemanager.h"
using namespace std;

//Summary: 输出销售经理的基本信息
//
//Parameters:
//    None
//Return: None
//Detail: 与employee类的函数相比, 增加了岗位信息与销售额
void salemanager::printInfo()
{
    cout<<"岗位\t\t编号\t姓名\t等级\t月薪\t销售额\t提成率"<<endl;
    cout<<"销售经理\t"<<getEmpNo()<<"\t"<<getName()<<"\t"<<getGrade()<<"\t"<<getAccumPay()<<"\t"<<getSales()<<"\t"<<getCommissionRate()<<endl;
    //TODO: 列出该经理的下属销售员编号
    cout<<"下属销售员编号: "<<endl;
    node<int> *node;
    node = salesmanNoList.getFirst();
    if (node != NULL)
    {

```

```

        do
        {
            cout<<node->data<<"\t";
            node = salesmanNoList.getNext();
        }while(node != NULL);
        cout<<endl;
    }
    else
    {
        cout<<"无"<<endl;
    }
}

//Summary: 计算工资
//
//Parameters:
//    None
//Return: int 工资的值
//Detail: 销售经理工资
int salemanager::PAY()
{
    int pay;
    pay = getFixedPay() + (int)(getCommissionRate() * getSales());
    setAccumPay(pay);
    return pay;
}

//Summary: 调级函数
//
//Parameters:
//    gradeUp 级别的变化值
//Return: bool 是否成功
//Detail: 每升一级固定工资增加, 提成比率增加.5%
bool salemanager::promote(int gradeUp)
{
    int grade_tmp = getGrade();
    int fixed_pay_tmp;
    double rate_tmp;
    if(grade_tmp + gradeUp > 0)
    {
        grade_tmp += gradeUp;
        fixed_pay_tmp = FIXED_PAY_SALEMANAGER + PROMOTE_PAY_SALEMANAGER * (grade_tmp
- 1);
        rate_tmp = RATE_SALEMANAGER + PROMOTE_RATE_SALEMANAGER * (grade_tmp - 1);
    }
}

```

```

        setFixedPay(fixed_pay_tmp);
        setCommissionRate(rate_tmp);
        setGrade(grade_tmp);
        PAY();
        return true;
    }
    return false;
}

//Summary: 无参构造函数
//
//Parameters:
//    None
//Return: None
//Detail: 调用上级构造函数，默认设置等级为，
//    销售额为
salemanager::salemanager():manager(), salesman()
{
    setCommissionRate(RATE_SALEMANAGER);
    salesmanNoList.makeEmpty();

    //TODO: 调用工资计算
    PAY();
}

//Summary: 有参构造函数
//
//Parameters:
//    inputEmpNo: 输入的雇员编号
//    inputGrade: 输入的雇员等级
//    inputName: 输入的雇员姓名
//    inputSales: 该雇员的月销售额
//    inputSalesman: 下属的销售员编号
//    numberSalesman: 下属的销售员数目
//Return: None
//Detail: 调用上级构造函数，设置等级为输入等级，
//    工资由销售额决定
salemanager::salemanager(int inputEmpNo, int inputGrade, string inputName, int
inputSales, linkedlist<int> &inputNoList)
:manager(inputEmpNo, 1, inputName), salesman(inputEmpNo, 1, inputName, inputSales,
NULL)
{
    setCommissionRate(RATE_SALEMANAGER);

```

```

        //TODO: 使用参数初始化链表
        salesmanNoList.makeEmpty();
        salesmanNoList = inputNoList;

        //TODO: 调级与工资计算
        promote(inputGrade - 1);
    }

//Summary: 析构函数
//
//Parameters:
//    None
//Return: None
salemanager::~~salemanager()
{
    //TODO: 清理销售员列表
    salesmanNoList.makeEmpty();
}

//Summary: 更改销售额函数
//
//Parameters:
//    inputSales: 新增的销售额
//Return: None
//Detail: 输入可以为负数，代表负增长或销售员离开
void salemanager::changeSales(int inputSales)
{
    int sales = getSales();
    sales += inputSales;
    setSales(sales);

    //TODO: 工资计算
    PAY();
}

//Summary: 增加销售员函数
//
//Parameters:
//    empNo: 新增的销售员编号
//Return: None
//Detail: 不能重复添加，但不会检查该编号是否存在，是否是一个销售员
bool salemanager::addSalesman(int empNo)
{
    //TODO: 检查与添加操作

```

```

        if(salesmanNoList.find(empNo))
            return false;
        node<int> *d = new node<int>;
        d->data = empNo;
        salesmanNoList.insert_end(d);
        return true;
    }

//Summary: 删除销售员函数
//
//Parameters:
//      empNo: 删除的销售员编号
//Return: None
//Detail: 如链表中不存在该编号, 则返回false
bool salemanager::deleteSalesman(int empNo)
{
    node<int> *node;
    node = salesmanNoList.getFirst();
    if(node != NULL)
    {
        do
        {
            if(node->data == empNo)
            {
                salesmanNoList.remove(node);
                return true;
            }
            node = salesmanNoList.getNext();
        }while(node != NULL);
    }
    return false;
}

```

```

//Summary: 清空销售员函数
//
//Parameters:
//      None
//Return: None
//Detail: 清空销售员列表, 置销售额
void salemanager::clearSalesman()
{
    salesmanNoList.makeEmpty();
    setSales(0);
    PAY();
}

```

```

}

//Summary: 更新信息函数
//
//Parameters:
//      None
//Return: None
//Detail: 更新该雇员的个人信息（除编号外）
void salemanager::updateInfo()
{
    cout<<"输入编号为"<<getEmpNo()<<" 的销售经理的姓名、等级："<<endl;
    int tempGrade;
    string tempName;
    cin>>tempName;
    cin>>tempGrade;

    setName(tempName);
    //TODO: 调级与工资计算
    promote(tempGrade - getGrade());
}

//Summary: 流输入操作符重载函数
//
//Parameters:
//      input: 输入流
//      sm:    目标销售经理对象
//Return: istream& 返回输入流本身
//Detail: 重载函数首先给出输入提示，之后从流中输入目标对象的等级
//注:    未添加合法性判断，请勿输入非正数
istream& operator >> (istream& input, salemanager& sm)
{
    //cout<<"输入编号为"<<sm.getEmpNo()<<" 的销售经理的姓名、等级："<<endl;
    int tempEmpNo;
    int tempGrade;
    string tempName;
    double tempVal;
    int tempSales;
    int tempSalesmanSize;
    int tempSalesmanNo;

    input>>tempEmpNo;
    input>>tempName;

```



```

        input>>tempGrade;
        input>>tempVal;
        input>>tempSales;
        input>>tempVal;

        sm.setEmpNo(tempEmpNo);
        sm.setName(tempName);
        //sm.setSales(tempSales);
        //TODO: 调级与工资计算
        sm.promote(tempGrade - sm.getGrade());

        input>>tempSalesmanSize;
        sm.salesmanNoList.makeEmpty();
        node<int> *d;
        for(int i = 0; i < tempSalesmanSize; i++)
        {
            input>>tempSalesmanNo;
            d = new node<int>;
            d->data = tempSalesmanNo;
            sm.salesmanNoList.insert_end(d);
        }
        return input;
    }

//Summary: 流输出操作符重载函数
//
//Parameters:
//    output: 输出流
//    sm:     目标销售经理对象
//Return: ostream& 返回输出流本身
ostream& operator << (ostream &output, salemanager &sm)
{
    output<<endl;
    output<<sm.getEmpNo()<<"\t"<<sm.getName()<<"\t"<<sm.getGrade()<<"\t"

    <<sm.getAccumPay()<<"\t"<<sm.getSales()<<"\t"<<sm.getCommissionRate()<<endl;
    //TODO: 列出该经理的下属销售员编号
    //cout<<sm.salesmanNoList.size();
    output<<sm.salesmanNoList.size();
    if(sm.salesmanNoList.size() == 0)
        return output;

    node<int> *node;
    node = sm.salesmanNoList.getFirst();

```

```

        if(node != NULL)
        {
            do
            {
                output<<" "<<node->data;
                node = sm.salesmanNoList.getNext();
            }while(node != NULL);
        }
        return output;
    }
}

/*****
FileName:salemanager.cpp
Copyright: Tsinghua University
Author: Huaizhou Tao
Date:2014-10-02
Description: salesman类的定义

*****/
#include <iostream>
#include "salesman.h"
#include "salemanager.h"
using namespace std;

//Summary: 输出销售员的基本信息
//
//Parameters:
//      None
//Return: None
//Detail: 与employee类的函数相比, 增加了岗位信息与销售额
void salesman::printInfo()
{
    cout<<"岗位\t编号\t姓名\t上司\t等级\t月薪\t销售额\t提成率"<<endl;

    int bossEmpNo;
    if(getBoss() == NULL)
        bossEmpNo = -1;
    else
        bossEmpNo = getBoss()->getEmpNo();

    cout<<"销售\t"<<getEmpNo()<<"\t"<<getName()<<"\t"<<bossEmpNo<<"\t"

        <<getGrade()<<"\t"<<getAccumPay()<<"\t"<<getSales()<<"\t"<<getCommissionRate()
    <<endl;
}

```

```

}

//Summary: 计算工资
//
//Parameters:
//      None
//Return: int 工资的值
//Detail: 销售工资
int salesman::PAY()
{
    int pay;
    pay = (int)(getSales() * getCommissionRate());
    setAccumPay(pay);
    return pay;
}

//Summary: 调级函数
//
//Parameters:
//      gradeUp 级别的变化值
//Return: bool 是否成功
//Detail: 每升一级提成比率增加%
bool salesman::promote(int gradeUp)
{
    int grade_tmp = getGrade();
    double rate_tmp;
    if(grade_tmp + gradeUp > 0)
    {
        grade_tmp += gradeUp;
        rate_tmp = RATE_SALESMAN + PROMOTE_RATE_SALESMAN * (grade_tmp - 1);

        setCommissionRate(rate_tmp);
        setGrade(grade_tmp);
        PAY();
        return true;
    }
    return false;
}

//Summary: 无参构造函数
//
//Parameters:
//      None
//Return: None

```

```

//Detail: 调用上级构造函数，默认设置等级为，
//      销售额与工资为
salesman::salesman():employee(0,1,0,"")
{
    sales = 0;
    commissionRate = RATE_SALESMAN;
    boss = NULL;
    //TODO: 调级与工资计算
    PAY();
}

//Summary: 有参构造函数
//
//Parameters:
//      inputEmpNo: 输入的雇员编号
//      inputGrade: 输入的雇员等级
//      inputName: 输入的雇员姓名
//      inputSales: 该雇员的月销售额
//      inputBoss: 该雇员的上司
//Return: None
//Detail: 调用上级构造函数，设置等级为输入等级，
//      工资由销售额决定
salesman::salesman(int inputEmpNo, int inputGrade, string inputName, int inputSales,
salemanager * inputBoss):employee(inputEmpNo, 1, 0, inputName)
{
    sales = inputSales;
    commissionRate = RATE_SALESMAN;

    boss = inputBoss;
    if(boss != NULL)
    {
        boss->changeSales(sales);
        //TODO: boss添加该销售员的操作
        boss->addSalesman(getEmpNo());
    }

    //TODO: 调级与工资计算
    promote(inputGrade - 1);
}

//Summary: 析构函数
//
//Parameters:
//      None

```

```

//Return: None
salesman::~salesman()
{
    if(boss !=NULL)
    {
        boss->changeSales(-sales);
        //TODO: boss删除该销售员的操作
        boss->deleteSalesman(getEmpNo());
    }
}

//Summary: 设置销售额函数
//
//Parameters:
//    inputSales: 新的销售额
//Return: None
//Detail: 因为涉及到上级的操作，不再使用内联函数
void salesman::setSales(int inputSales)
{
    if(boss != NULL)
    {
        boss->changeSales(inputSales - sales);
    }
    sales = inputSales;

    //TODO: 调级与工资计算
    PAY();
}

//Summary: 设置上级函数
//
//Parameters:
//    inputBoss: 新的上级
//Return: None
//Detail: 变更上级时，要同时变更上级的销售额
void salesman::setBoss(salemanager *inputBoss)
{
    if(boss != NULL)
    {
        boss->changeSales(-sales);
        //TODO: 原有boss删除该销售员的操作
        boss->deleteSalesman(getEmpNo());
    }
    if(inputBoss != NULL)

```

```

    {
        inputBoss->changeSales(sales);
        //TODO: 新boss添加该销售员的操作
        inputBoss->addSalesman(getEmpNo());
    }
    boss = inputBoss;
}

//Summary: 更新信息函数
//
//Parameters:
//    None
//Return: None
//Detail: 更新该雇员的个人信息（除编号外）
void salesman::updateInfo()
{
    //不可输入boss编号，使用另外的函数在database中进行上下级关系设置
    cout<<"输入编号为"<<getEmpNo()<<" 的销售员的姓名、等级与销售额："<<endl;
    int tempGrade;
    string tempName;
    cin>>tempName;
    cin>>tempGrade;
    cin>>sales;

    setName(tempName);
    //TODO: 由于等级变化，进行调级与工资计算
    promote(tempGrade - getGrade());
}

//Summary: 流输入操作符重载函数
//
//Parameters:
//    input: 输入流
//    s:      目标销售员对象
//Return: istream& 返回输入流本身
//Detail: 重载函数首先给出输入提示，之后从流中输入目标对象的等级与销售额
//注:      未添加合法性判断，请勿输入非正数；目前主函数的数据结构还未设计，
//          所以无法通过输入编号给销售员指定上级，该功能留待进一步完善
istream& operator >> (istream& input, salesman& s)
{
    //不可输入boss编号，使用另外的函数在database中进行上下级关系设置
    //cout<<"输入编号为"<<s.getEmpNo()<<" 的销售员的姓名、等级与销售额："<<endl;
    int tempEmpNo;
    int tempGrade;

```

```

    string tempName;
    double tempVal;

    input>>tempEmpNo;
    input>>tempName;
    input>>tempGrade;
    input>>tempVal;
    input>>s.sales;
    input>>tempVal;

    s.setEmpNo(tempEmpNo);
    s.setName(tempName);
    //TODO: 调级与工资计算
    s.promote(tempGrade - s.getGrade());
    //TODO: 通过输入编号给销售员指定上级

    return input;
}

//Summary: 流输出操作符重载函数
//
//Parameters:
//    output: 输出流
//    s:      目标销售员对象
//Return: ostream& 返回输出流本身
ostream& operator << (ostream &output, salesman &s)
{
    int bossEmpNo;
    if(s.getBoss() == NULL)
        bossEmpNo = -1;
    else
        bossEmpNo = s.getBoss()->getEmpNo();

    output<<endl<<s.getEmpNo()<<' \t'<<s.getName()<<' \t'
        <<s.getGrade()<<' \t'<<s.getAccumPay()<<' \t'
        <<s.getSales()<<' \t'<<s.getCommissionRate()<<' \t'
        <<bossEmpNo;

    return output;
}

/*****
FileName:technician.cpp
Copyright: Tsinghua University

```

Author: Huaizhou Tao

Date:2014-10-02

Description: technician类的定义

```

*****/
#include <iostream>
#include "technician.h"
using namespace std;

//Summary: 输出技术员的基本信息
//
//Parameters:
//      None
//Return: None
//Detail: 与employee类的函数相比, 增加了岗位信息与工作时间
void technician::printInfo()
{
    cout<<"岗位\t编号\t姓名\t等级\t月薪\t工时\t时薪"<<endl;
    cout<<"技术\t"<<getEmpNo()<<"\t"<<getName()<<"\t"<<getGrade()<<"\t"
        <<getAccumPay()<<"\t"<<getWorkHour()<<"\t"<<getWage()<<endl;
}

//Summary: 计算工资
//
//Parameters:
//      None
//Return: int 工资的值
//Detail: 技术工资
int technician::PAY()
{
    int pay;
    pay = getWorkHour() * getWage();
    setAccumPay(pay);
    return pay;
}

//Summary: 调级函数
//
//Parameters:
//      gradeUp 级别的变化值
//Return: bool 是否成功
//Detail: 每升一级, 时薪增加
bool technician::promote(int gradeUp)
{

```



```

    int grade_tmp = getGrade();
    int wage_tmp;
    if(grade_tmp + gradeUp > 0)
    {
        grade_tmp += gradeUp;
        wage_tmp = WAGE_TECHNICIAN + PROMOTE_WAGE_TECHNICIAN * (grade_tmp - 1);

        setWage(wage_tmp);
        setGrade(grade_tmp);
        PAY();
        return true;
    }
    return false;
}

```

//Summary: 无参构造函数

//

//Parameters:

// None

//Return: None

//Detail: 调用上级构造函数，默认设置等级为，

// 工作时间与工资为

technician::technician():employee(0,1,0,"")

```

{
    workHour = 0;
    wage = WAGE_TECHNICIAN;
    //TODO: 调级与工资计算
    PAY();
}

```

//Summary: 有参构造函数

//

//Parameters:

// inputEmpNo: 输入的雇员编号

// inputGrade: 输入的雇员等级

// inputName: 输入的雇员姓名

// inputWorkHour: 该雇员的月工作时间

//Return: None

//Detail: 调用上级构造函数，设置等级为输入等级，

// 工资由工作时间决定

```

technician::technician(int inputEmpNo, int inputGrade, string inputName, int
inputWorkHour):employee(inputEmpNo, 1, 0, inputName)
{
    workHour = inputWorkHour;
}

```

```

        wage = WAGE_TECHNICIAN;
        //TODO: 调级与工资计算
        promote(inputGrade - 1);
    }

//Summary: 析构函数
//
//Parameters:
//      None
//Return: None
technician::~technician()
{

}

//Summary: 更新信息函数
//
//Parameters:
//      None
//Return: None
//Detail: 更新该雇员的个人信息（除编号外）
void technician::updateInfo()
{
    cout<<"输入编号为"<<getEmpNo()<<" 的技术员的姓名、等级与工作时间："<<endl;
    int tempGrade;
    string tempName;
    cin>>tempName;
    cin>>tempGrade;
    cin>>workHour;

    setName(tempName);
    //TODO: 调级与工资计算
    promote(tempGrade - getGrade());
}

//Summary: 流输入操作符重载函数
//
//Parameters:
//      input: 输入流
//      t:      目标技术员对象
//Return: istream& 返回输入流本身
//Detail: 重载函数首先给出输入提示，之后从流中输入目标对象的等级与工作时间
//注:      未添加合法性判断，请勿输入非正数；
istream& operator >> (istream& input, technician& t)

```

```

{
    //cout<<"输入编号为"<<t.getEmpNo()<<" 的技术员的姓名、等级与工作时间："<<endl;
    int tempEmpNo;
    int tempGrade;
    string tempName;
    int tempPay;
    int tempWage;

    input>>tempEmpNo;
    input>>tempName;
    input>>tempGrade;
    input>>tempPay;
    input>>t.workHour;
    input>>tempWage;

    t.setEmpNo(tempEmpNo);
    t.setName(tempName);
    //TODO: 调级与工资计算
    t.promote(tempGrade - t.getGrade());
    return input;
}

```

//Summary: 流输出操作符重载函数

//

//Parameters:

// output: 输出流

// t: 目标技术员对象

//Return: ostream& 返回输出流本身

ostream& operator << (ostream &output, technician &t)

```

{
    output<<endl<<t.getEmpNo()<<"\t"<<t.getName()<<"\t"<<t.getGrade()<<"\t"
        <<t.getAccumPay()<<"\t"<<t.getWorkHour()<<"\t"<<t.getWage();
    return output;
}

```

/*****

FileName:userinterface.cpp

Copyright: Tsinghua University

Author: Huaizhou Tao

Date:2014-10-02

Description: 用户界面类的定义

*****/

#include <iostream>

```

#include "userinterface.h"
using namespace std;

//Summary: 构造函数
//
//Parameters:
//      None
//Return: None
//Detail:
userinterface::userinterface()
{
    emp_database = NULL;
}

//Summary: 析构函数
//
//Parameters:
//      None
//Return: None
//Detail:
userinterface::~userinterface()
{
    delete emp_database;
}

//Summary: 主要的交互界面
//
//Parameters:
//      None
//Return: bool 是否继续
//Detail:
bool userinterface::running()
{
    system("cls");
    cin.clear();
    cin.sync();

    if(emp_database == NULL)
    {
        emp_database = new database();
        try
        {
            emp_database->load();
        }
    }
}

```

```

        catch(FileException e)
        {
            if(e.mode == "open")
            {
                cout<<"以"<<e.type<<" 方式打开文件"<<e.filename<<" 时出错。"<<endl;
            }
            else
            {
                if(e.type == "read")
                {
                    cout<<"从文件"<<e.filename<<" 读取时出错。"<<endl;
                }
                else
                {
                    cout<<"向文件"<<e.filename<<" 写入时出错。"<<endl;
                }
            }
            return false;
        }
    }
}

```

```
welcome();
```

```

int input;
cin>>input;

```

```

if(!cin)
{
    cout<<"输入错误, 请重新输入."<<endl;
    pause();
    return true;
}

```

```

switch(input)
{
case 1:
    while(insertEmp());
    break;
case 2:
    while(deleteEmp());
    break;
case 3:
    while(searchEmp());
    break;
}

```

```

        case 4:
            while(updateEmp());
            break;
        case 5:
            while(sortEmp());
            break;
        case 6:
            while(setRelation());
            break;
        case 7:
            quit();
            return false;
        default:
            cout<<"输入错误, 请重新输入."<<endl;
    }

    pause();
    return true;
}

//Summary: 暂停函数
//
//Parameters:
//    None
//Return: None
//Detail:
void userinterface::pause()
{
    cin.clear();
    cin.sync();
    cout<<"按任意键继续:";
    getchar();
}

//Summary: 欢迎界面
//
//Parameters:
//    None
//Return: None
//Detail:
void userinterface::welcome()
{
    cout << "        犀利哥人事管理系统"<<endl
        << "\t1. 录入雇员信息"<<endl

```

```

        << "\t2. 删除雇员信息"<<endl
        << "\t3. 查找雇员信息"<<endl
        << "\t4. 更新雇员信息"<<endl
        << "\t5. 排序雇员信息"<<endl
        << "\t6. 设置销售部关系"<<endl
        << "\t7. 退出"<<endl;

    cout << "输入编号进行操作:";
}

//Summary: 查询界面
//
//Parameters:
//    None
//Return: bool 返回是否继续
//Detail:
bool userinterface::searchEmp()
{
    system("cls");

    cout << "        查找雇员信息"<<endl
        << "\t1. 按编号查找"<<endl
        << "\t2. 按姓名查找"<<endl
        << "\t3. 显示某类全部雇员"<<endl
        << "\t4. 返回上级"<<endl;

    cout << "输入编号进行操作:";

    int input;
    int num;
    int empNo;
    string empName;
    int empType;
    cin>>input;

    if(!cin)
    {
        cout<<"输入错误, 请重新输入."<<endl;
        pause();
        return true;
    }

    switch(input)
    {

```

case 1:

```
cout<<"          按编号查找雇员"<<endl;
cout<<"输入待查雇员编号: ";
cin>>empNo;
if(!cin)
{
    cout<<"输入错误, 请重新输入."<<endl;
    pause();
    return true;
}
num = emp_database->search(empNo);
if(num == 0)
    cout<<"没有找到结果."<<endl;
else
    cout<<"找到"<<num<<" 个结果."<<endl;
break;
```

case 2:

```
cout<<"          按姓名查找雇员"<<endl;
cout<<"输入待查雇员姓名: ";
cin>>empName;
if(!cin)
{
    cout<<"输入错误, 请重新输入."<<endl;
    pause();
    return true;
}
num = emp_database->search(empName);
if(num == 0)
    cout<<"没有找到结果."<<endl;
else
    cout<<"找到"<<num<<" 个结果."<<endl;
break;
```

case 3:

```
cout<<"          按类显示全部雇员"<<endl;
cout<<"输入雇员类别:"<<endl;
cout<<"1. 经理"<<endl
<<"2. 技术"<<endl
<<"3. 销售"<<endl
<<"4. 销售经理"<<endl
<<"0. 全部"<<endl;
cin>>empType;
if(!cin)
```



```

    {
        cout<<"输入错误, 请重新输入."<<endl;
        pause();
        return true;
    }
    num = emp_database->showAll((EmployeeType) empType);
    if(num == 0)
        cout<<"没有找到结果."<<endl;
    else
        cout<<"找到"<<num<<" 个结果."<<endl;
    break;
case 4:
    cout<<"返回上级菜单."<<endl;
    return false;
default:
    cout<<"输入错误, 请重新输入."<<endl;
    break;
}

pause();
return true;
}

//Summary: 插入界面
//
//Parameters:
//    None
//Return: bool 返回是否继续
//Detail:
bool userinterface::insertEmp()
{
    system("cls");

    cout<<"        录入雇员信息"<<endl;
    cout<<"输入雇员类别:"<<endl;
    cout<< "1. 经理"<<endl
    << "2. 技术"<<endl
    << "3. 销售"<<endl
    << "4. 销售经理"<<endl;

    int empType;
    cin>>empType;
    if(!cin)
    {

```

```

        cout<<"输入错误, 请重新输入."<<endl;
        pause();
        return true;
    }

    if(empType < 1 || empType > 4)
    {
        cout<<"输入错误, 请重新输入."<<endl;
        pause();
        return true;
    }

    emp_database->addEmployee((EmployeeType)empType);

    cout<<"是否继续?(y/n)";
    char input;
    cin>>input;
    if(input == 'y' || input == 'Y')
        return true;
    else
        return false;
}

//Summary: 删除界面
//
//Parameters:
//    None
//Return: bool 返回是否继续
//Detail:
bool userinterface::deleteEmp()
{
    system("cls");

    cout<<"        删除雇员信息"<<endl;
    cout<<"输入雇员编号:"<<endl;

    int empNo;
    cin>>empNo;
    if(!cin)
    {
        cout<<"输入错误, 请重新输入."<<endl;
        pause();
        return true;
    }
}

```

```

        if(empNo <= 0)
        {
            cout<<"输入错误, 请重新输入."<<endl;
            pause();
            return true;
        }

        if(emp_database->deleteEmployee(empNo))
        {
            cout<<"删除结束. "<<endl;
        }
        else
        {
            cout<<"未找到目标雇员. "<<endl;
        }

        cout<<"是否继续?(y/n)";
        char input;
        cin>>input;
        if(input == 'y' || input == 'Y')
            return true;
        else
            return false;
    }

//Summary: 更新界面
//
//Parameters:
//      None
//Return: bool 返回是否继续
//Detail:
bool userinterface::updateEmp()
{
    system("cls");

    cout<<"      更新雇员信息"<<endl;
    cout<<"输入雇员编号:"<<endl;

    int empNo;
    cin>>empNo;
    if(!cin)
    {
        cout<<"输入错误, 请重新输入."<<endl;
    }
}

```

```

        pause();
        return true;
    }

    if(empNo <= 0)
    {
        cout<<"输入错误, 请重新输入."<<endl;
        pause();
        return true;
    }

    if(emp_database->updateEmployee(empNo))
    {
        cout<<"更新结束. "<<endl;
    }
    else
    {
        cout<<"未找到目标雇员. "<<endl;
    }

    cout<<"是否继续?(y/n)";
    char input;
    cin>>input;
    if(input == 'y' || input == 'Y')
        return true;
    else
        return false;
}

//Summary: 排序界面
//
//Parameters:
//      None
//Return: bool 返回是否继续
//Detail:
bool userinterface::sortEmp()
{
    system("cls");

    cout << "      雇员信息排序"<<endl
        << "\t1. 按编号排序"<<endl
        << "\t2. 按工资排序"<<endl
        << "\t3. 返回上级"<<endl;

```

```
cout << "输入编号进行操作:";

int key;
int direction;
int empType;
cin>>key;
if(!cin)
{
    cout<<"输入错误, 请重新输入."<<endl;
    pause();
    return true;
}

switch(key)
{
case 1:
    break;
case 2:
    break;
case 3:
    cout<<"返回上级菜单."<<endl;
    return false;
default:
    cout<<"输入错误, 请重新输入."<<endl;
    pause();
    return true;
}

cout<<"输入雇员类别:"<<endl;
cout<< "1. 经理"<<endl
<< "2. 技术"<<endl
<< "3. 销售"<<endl
<< "4. 销售经理"<<endl;

cin>>empType;
if(!cin)
{
    cout<<"输入错误, 请重新输入."<<endl;
    pause();
    return true;
}

if(empType < 1 || empType > 4)
{
```

```

        cout<<"输入错误, 请重新输入."<<endl;
        pause();
        return true;
    }

    cout<<"输入排序方向:"<<endl;
    cout<< "1. 升序"<<endl;
    << "2. 降序"<<endl;

    cin>>direction;
    if(!cin)
    {
        cout<<"输入错误, 请重新输入."<<endl;
        pause();
        return true;
    }

    if(direction < 1 || direction > 2)
    {
        cout<<"输入错误, 请重新输入."<<endl;
        pause();
        return true;
    }

    switch(key)
    {
    case 1:
        emp_database->sortByNo((EmployeeType)empType, (direction == 1)?-1:1);
        break;
    case 2:
        emp_database->sortByPay((EmployeeType)empType, (direction == 1)?-1:1);
        break;
    }

    pause();
    return true;
}

//Summary: 退出界面
//
//Parameters:
//      None
//Return: None

```

```

//Detail:
void userinterface::quit()
{
    cout<<"是否保存数据? (y/n):";
    char input;
    cin >> input;
    if(input == 'y' || input == 'Y')
    {
        try
        {
            emp_database->save();
        }
        catch(FileException e)
        {
            if(e.mode == "open")
            {
                cout<<"以"<<e.type<<" 方式打开文件"<<e.filename<<" 时出错。"<<endl;
            }
            else
            {
                if(e.type == "read")
                {
                    cout<<"从文件"<<e.filename<<" 读取时出错。"<<endl;
                }
                else
                {
                    cout<<"向文件"<<e.filename<<" 写入时出错。"<<endl;
                }
            }
        }
    }

    cout<<"谢谢使用, 再见!"<<endl;
}

//Summary: 设置关系界面
//
//Parameters:
//      None
//Return: bool 返回是否继续
//Detail:
bool userinterface::setRelation()
{
    system("cls");
}

```

```

cout << "          设置销售部关系"<<endl;

cout << "输入销售员编号:";

int saleNo;
int saleMngNo;

cin>>saleNo;

if(!cin)
{
    cout<<"输入错误, 请重新输入."<<endl;
    pause();
    return true;
}

cout << "输入销售经理编号:";
cin>>saleMngNo;

if(!cin)
{
    cout<<"输入错误, 请重新输入."<<endl;
    pause();
    return true;
}

bool result = emp_database->setRelation(saleNo, saleMngNo);

if(result)
{
    cout<<"设置成功."<<endl;
}
else
{
    cout<<"设置失败, 编号不存在."<<endl;
}

cout<<"是否继续?(y/n)";
char input;
cin>>input;
if(input == 'y' || input == 'Y')
    return true;
else

```



```
    return false;
}
```

本章小结

本章主要介绍了 C++ 的主要编程工具。包括函数模板、类模板、例外处理、命名空间、标准类库，以及标准模板库（STL）。其中，标准模板库是在函数模板和类模板的基础上建立一套泛型编程的工具库，极大地提高了 C++ 的程序设计的重用性。但 STL 涉及内容比较多，而且与后续课程《数据结构与算法分析》的知识点密切相关。因此，在本章中只对 STL 的概念做了一些简单介绍。主要讨论了 C++ 标准模板库（STL）中常用的四大组件：容器、迭代器、算法和函数对象。其中常用的顺序容器 `vector`、`list`、`deque` 是重点；对迭代器的理解是应用 STL 的关键和难点，可以类比指针的概念会更加容易理解迭代器的概念；STL 提供的算法集合强大而且丰富，这些算法都具有非常高的通用性和运行效率，在实践中要加强对各类算法的理解和应用；另外函数对象常用作算法的参数，表示执行操作的方式。

课后习题

1. 什么是程序中标识符作用范围？有哪些种类的作用范围？阐述命名空间的作用？
2. 解释为什么在 c++ 每个程序前面需要加一条：`using namespace std;` 语句？
3. 求 C++ 中的异常处理机制意义，作用是什么？
4. 当在 `try` 块中抛出异常后，程序最后是否回到 `try` 块中继续执行后面的语句？
5. 什么叫抛出异常？`catch` 可以获取什么异常参数？是根据异常参数的类型还是根据参数的值处理异常？请编写测试程序验证。
6. 为什么 C++ 要求资源的取得放在构造函数中，而资源的释放在析构函数？以 `String` 类为例，在 `String` 类的构造函数中使用 `new` 分配内存。如果操作不成功，则用 `try` 语句触发一个 `char` 类型异常，用 `catch` 语句捕获该异常。同时将异常处理机制与其他处理方式对内存分配失败这一异常进行处理对比，体会异常处理机制的优点。
7. 简述函数模板生成函数的过程。
8. 简述类模板生成对象的过程。
9. 简述函数模板与模板函数、类模板与模板类的区别？
10. 设计一个函数模板，其中包括数据成员 `T a[n]` 以及对其进行排序的成员函数 `sort()`，模板参数 `T` 可实例化成字符串。
11. 设计一个类模板，其中包括数据成员 `T a[n]` 以及在其中进行查找数据元素的函数 `int search(T)`，模板参数 `T` 可实例化成字符串。